MORSE-BASED MODULAR HOMOLOGY FOR EVOLVING SIMPLICIAL COMPLEXES

A PREPRINT

Anqiao Ouyang
Mount Boucherie Secondary School
West Kelowna, British Columbia, Canada

Department of Aerospace Engineering
San Diego State University
San Diego, California, USA
aouyang@sdsu.edu

October 12, 2026

ABSTRACT

We study the maintenance of the Betti numbers over $\mathbb{Z}_2$ of a simplicial complex under insertions and deletions of simplices. For each dimension we maintain a reduced matrix $R = BV$ together with the change of basis V, a dynamic reduced basis. An inserted simplex is reduced against the existing pivots; a deleted simplex is removed from every basis chain that contains it, by adding either a cycle or the column with the smallest pivot, the rank-only form of known updates for zigzag persistence and warm-started reductions. We prove that the Betti numbers are exact after every edit, for every order of the cells. In the order given by a collapse, every gradient pair is an apparent pair, and a build performs column additions only in the columns of critical cells, where they coincide with the reduction of the Morse complex. The cost of an edit is output-sensitive, and on explicit families every toggle of a single simplex costs $\Omega(n)$, so no bound in terms of the size of an edit holds. On five workload families with up to 3,932,162 simplices, the method reported the correct Betti numbers in every checked state. Where edits create and destroy only small cycles, its latency depends little on the size of the complex: on 955,059 simplices it is 88.6–89.9 times faster than the better static recomputation, a gain due to the dynamic basis itself. Growth-triggered rebuilds and the Morse order add further gains where edits change the topology and the basis fills in. Where edits create cycles that span the complex, the cost grows linearly, as the lower bound predicts, and the tail latency can exceed that of recomputation.

Keywords Simplicial Homology, Betti Numbers, Dynamic Homology Computation, Discrete Morse Theory, Boundary Matrix Reduction, Computational Algebraic Topology

1 Introduction

The Betti numbers $\beta_0, \beta_1, \beta_2, \beta_3$ of a simplicial complex count its connected components, independent loops, enclosed voids and, in dimension three, independent 3-cycles; over a field they are determined by the ranks of the boundary matrices [17, 24, 35]. In many uses the complex is not fixed. Adaptive meshes are refined and coarsened by local moves [37], cavities are carved into a volume mesh and refilled, holes are opened in a surface and closed again, and the proximity complex of a point cloud changes as the points move [6]. After each modification one wants to know whether it changed the topology. Recomputing the Betti numbers from scratch answers this, but it reads the whole complex after every modification, however small the modification is.

Over a field, $\beta_k = n_k - r_k - r_{k+1}$, where n_k is the number of k-simplices and r_k the rank of the k-th boundary matrix B_k. Maintaining Betti numbers therefore means maintaining ranks, and the problem has more structure than

general dynamic rank: the boundary of a simplex does not change while the simplex exists, so each atomic edit inserts or deletes one column of one boundary matrix, and rows appear and disappear only while they are zero (Observation 2.1). Insertions alone are handled by reducing the new column against the existing pivots: each inserted simplex either creates a cycle or destroys one [11]. Deletions are the difficulty. After reduction, a column is a sum of original columns; when one of these is deleted, every reduced column that absorbed it becomes invalid, and the reduced matrix alone does not record which columns these are. This paper maintains that record and studies what it costs.

Contributions.

(i) *Exact dynamic reduced basis maintenance.* For each dimension we maintain a reduced matrix $R = B_k V$ together with the change of basis V and, for every simplex, the columns of V that contain it. Insertion is the standard incremental reduction; a deletion corrects every basis chain that contains the deleted simplex, by a cycle (Case A) or by the column with the smallest pivot (Case B). We prove that the Betti numbers over $\mathbb{Z}_2$ are exact after every edit, for every order of the cells and in every dimension, and that initialisation and rebuilds by the static reduction with clearing produce a valid state (Theorem 1 and Proposition 1.1). The deletion step is the rank-only specialisation of the deletion updates used for zigzag persistence and warm-started $R = DV$ factorisations [8, 22, 28]; we give a self-contained proof.

(ii) *A Morse-induced order and its properties.* Ordering the cells by reverse removal time in a collapse makes every gradient pair an apparent pair (Lemma 2, close to known results), so a build performs column additions only in the columns of critical cells (Corollary 2.1), and on those columns it performs exactly the reduction of the Morse complex (Corollary 2.2). Between rebuilds the order is a heuristic, and we measure its effect.

(iii) *Output-sensitive costs and the limits of locality.* The cost of an edit is stated in terms of footprints and reduction chains (Proposition 3). On two explicit families, a cycle graph and a triangulated sphere, every toggle of one simplex costs this data structure $\Omega(n)$ operations (Proposition 4), so no bound in terms of the size of an edit holds.

(iv) *An evaluation that identifies where the method is effective.* On five workload families with up to 3,932,162 simplices we compare the method with static recomputation in two orders and with GUDHI [30], and separate the contributions of the basis, the order and rebuilding by ablations (Section 6). Every Betti vector reported by an exact method was checked against a reference computation, and none disagreed. Where edits create and destroy only small cycles, the latency hardly depends on the size of the complex: on the largest refinement instance (955,059 simplices) every variant is 88.6–89.9 times faster than the better static baseline, a gain due to the basis itself. Where edits change the topology and the basis fills in, growth-triggered rebuilds and the Morse order together give the fastest variant. Where edits create cycles that span the complex, the cost grows linearly, as Proposition 4 predicts.

Scope. All statements are over $\mathbb{Z}_2$; we say nothing about integer homology or torsion. The maintained basis yields ranks and a basis of each cycle space, but no persistence pairing. We prove no bound on the cost of an update in terms of its size, and we make no sparsity claim for the state between rebuilds.

Outline. Section 2 states the problem and gives an overview of the algorithm. Section 3 defines the dynamic reduced basis and proves its exactness, Section 4 introduces the Morse order and its properties, and Section 5 analyses the cost of an update and proves that no locality bound holds. Section 6 reports the experiments, Section 7 discusses limitations and when to use which method, Section 8 reviews related work, and Section 9 concludes.

2 Problem and algorithm overview

2.1 Setting

Let K be a finite simplicial complex of dimension d. A k-simplex is a set of $k+1$ vertices; K_k is the set of k-simplices of K and $n_k = |K_k|$. We identify the chain group $C_k = C_k(K; \mathbb{Z}_2)$ with the set of finite subsets of K_k, addition being symmetric difference. The boundary $\partial_k \colon C_k \to C_{k-1}$ sends a simplex to the set of its facets (codimension-one faces); B_k is its matrix, with rows indexed by K_{k-1} and columns by K_k. We write $Z_k = \ker \partial_k$, $r_k = \operatorname{rank} B_k$ for $1 \le k \le d$, and $r_0 = r_{d+1} = 0$. By rank–nullity, $\dim Z_k = n_k - r_k$ and $\dim \operatorname{im} \partial_{k+1} = r_{k+1}$, so

$$\beta_k \;=\; n_k - r_k - r_{k+1} \qquad (0 \le k \le d). \tag{1}$$

Edits and updates. An *atomic edit* is either INSERT(σ), for a simplex $\sigma \notin K$ all of whose facets lie in K, or DELETE(σ), for a simplex $\sigma \in K$ that has no proper coface in K. Each atomic edit leaves a simplicial complex. An *update* u_t is a finite sequence of atomic edits (a batch); its *size* is $s_t = |u_t|$. The task is to output $\beta_0, \dots, \beta_d$ of K_t after every update t.

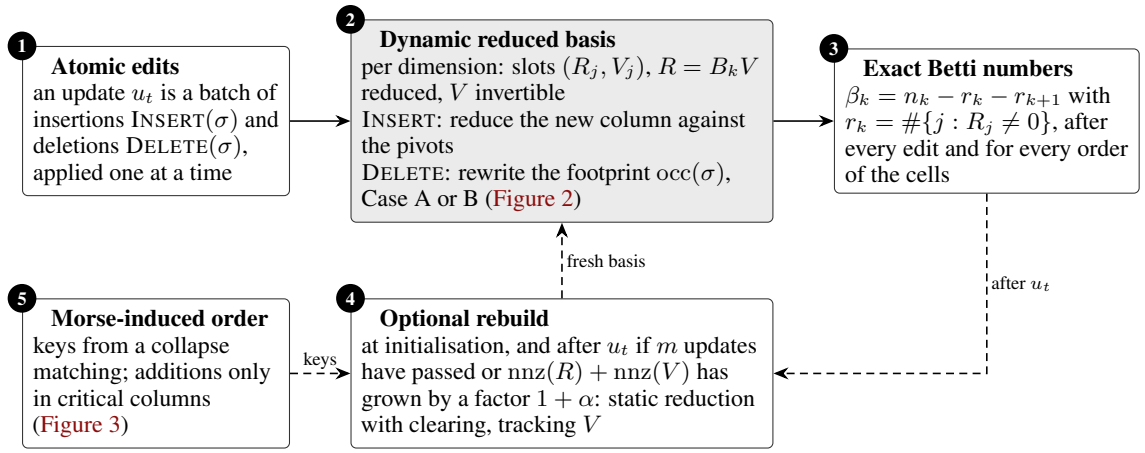


Figure 1: The five parts of the method. *Solid arrows:* each atomic edit of an update (1) changes the dynamic reduced basis of one dimension (2, Section 3), from which the Betti numbers are read off (3, Theorem 1); this path is exact for every order of the cells. *Dashed arrows:* a rebuild (4, Proposition 1.1) recomputes the basis by static reduction in the Morse-induced order (5, Section 4); this path changes only the cost. The ablations of Section 6.1 vary the dashed path: drb-id builds once in insertion order and never rebuilds, mmhm-norb builds once in the Morse order, and drb-id-rb rebuilds with the policy of mmhm but in insertion order.

Observation 2.1 (Structural fact)**.** *Let σ be a k-simplex. While $\sigma \in K$, its boundary $\partial\sigma$ does not change. Inserting σ appends the column $\partial\sigma$ to B_k and appends a zero row (indexed by σ) to B_{k+1}; deleting σ removes its column from B_k and removes its row from B_{k+1}, and that row is zero. No other boundary matrix changes, and among $n_0, \ldots, n_d$ only n_k changes.*

Proof. The boundary of a simplex is determined by its vertex set. When σ is inserted it has no coface: a coface would already be in K, and K is closed under faces while $\sigma \notin K$. A deleted simplex has no coface by the precondition of DELETE. In both cases the row σ of B_{k+1} is zero. □

Consequently, for each $k \geq 1$ the problem is to maintain $r_k = \mathrm{rank}\, B_k$ under *column insertion and column deletion*, where rows appear and disappear only while they are zero; adding or removing a zero row changes no rank. The vertex dimension $k = 0$ needs no matrix: $\partial_0 = 0$.

2.2 The algorithm at a glance

For each dimension $k \geq 1$ we store a reduced matrix $R = B_k V$ together with the invertible change of basis V, and for every k-simplex σ the set $\mathrm{occ}(\sigma)$ of columns of V that contain it, the *footprint* of σ (Figure 1). An insertion reduces the new column against the existing pivots, as in the standard persistence algorithm [18, 39]. A deletion rewrites every column in the footprint of the deleted simplex: if one of them is a cycle, that cycle is added to the others (Case A); otherwise the column with the smallest pivot is added to the others (Case B, Figure 2). Section 3 proves that this keeps the Betti numbers exact after every atomic edit, for every order of the cells. The order affects only the cost. When the structure is built or rebuilt we compute an acyclic matching by collapses and order the cells by reverse removal time (Section 4 and Figure 3); under this order the static reduction performs all its column additions in the columns of critical cells. Between rebuilds the order is a heuristic for a sparse basis. Section 5 states the cost of an edit in output-sensitive terms and shows that it admits no bound in terms of the size of the edit. Coefficients are in $\mathbb{Z}_2$ throughout. The proofs hold in every dimension; the implementation and the experiments are restricted to $\dim K \leq 3$.

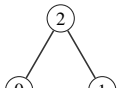
3 The dynamic reduced basis

Every cell $x \in K$ carries an *order key* $\mathrm{ord}(x)$; keys of distinct cells are distinct. For a non-zero chain c, $\mathrm{low}(c)$ is the cell of c with the largest key. Keys do not change between rebuilds (Section 4); an inserted cell receives a key larger than every key in use. Nothing in this section depends on how the keys are chosen; Section 4 chooses them.

Definition 3.1 (DRB)**.** Let $k \geq 1$. A *dynamic reduced basis* of B_k is a finite family of *slots* $j \in J_k$, each holding a pair (R_j, V_j) with $V_j \in C_k$ and $R_j \in C_{k-1}$, such that

(I1) $R_j = \partial_k V_j$ for every $j \in J_k$;

(a) Case A: $\sigma = 01$ lies on a cycle



$\beta = (1, 1)$

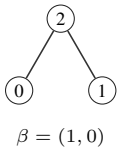
$V =$

	s_1	s_2	s_3
01	1	0	1
02	0	1	1
12	0	0	1

$R =$

	s_1	s_2	s_3
0	1	1	0
1	1	0	0
2	0	1	0

$O = \mathrm{occ}(\sigma) = \{s_1, s_3\}$ (the 1s in the shaded row);
$R_{s_3} = 0$, so $Z = \{s_3\}$ and $z = s_3$.
$V_{s_1} \leftarrow V_{s_1} + V_{s_3}$; remove slot s_3 and row σ.



$\beta = (1, 0)$

$V =$

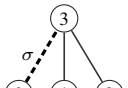
	s_1	s_2
02	**1**	1
12	**1**	0

$R =$

	s_1	s_2
0	1	1
1	1	0
2	0	1

Why a cycle. $\partial V_{s_3} = R_{s_3} = 0$, so the addition removes σ from V_{s_1} and leaves every R_j, hence every low, as it was. n_1 drops and $r_1 = 2$ stays: β_1 falls by one.

(b) Case B: $\sigma = 03$ lies on no cycle



$\beta = (1, 0)$

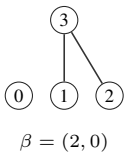
$V =$

	s_1	s_2	s_3
03	1	1	1
13	0	1	0
23	0	0	1

$R =$

	s_1	s_2	s_3
0	1	1	1
1	0	1	0
2	0	0	1
3	1	0	0

$O = \{s_1, s_2, s_3\}$ and every $R_j \neq 0$, so $Z = \emptyset$; the lows are 3, 1, 2, and $p = s_2$ has the smallest.
$(R_j, V_j) \leftarrow (R_j + R_{s_2}, V_j + V_{s_2})$ for $j = s_1, s_3$;
remove slot s_2 and row σ.



$\beta = (2, 0)$

$V =$

	s_1	s_3
13	**1**	**1**
23	0	1

$R =$

	s_1	s_3
0	**0**	**0**
1	**1**	**1**
2	0	1
3	1	0

Why the smallest low. Adding $R_{s_1} = \{0, 3\}$, whose low 3 is the largest, would instead give $\{1, 3\}$ and $\{2, 3\}$: two lows 3, violating (I2). R_{s_2} lies entirely above the other lows, so adding it moves none. n_1 and r_1 both drop: β_0 rises by one.

Figure 2: The two cases of DELETE(σ), worked on graphs ($k = 1$). Columns are slots. The rows of V are edges; the rows of R are vertices in key order from top to bottom, so the low of a column is its lowest non-zero entry (boxed). The shaded row of V is σ, and its 1s mark the footprint O. Bold entries are those changed by the update; $\beta = (\beta_0, \beta_1)$. (a) A footprint slot is a cycle ($R_{s_3} = 0$); adding it to the other footprint slot clears σ from V and leaves R unchanged. (b) No footprint slot is a cycle; the slot with the smallest low is added to the others and removed, and no low moves. Every entry is recomputed and checked by `reference/figure_examples.py`.

(I2) the non-zero R_j have pairwise distinct lows;
(I3) $(V_j)_{j \in J_k}$ is a basis of C_k; in particular $|J_k| = n_k$.

Two index structures are derived from the slots: $\mathrm{pivot}(\ell) = j$ if $R_j \neq 0$ and $\mathrm{low}(R_j) = \ell$ (well defined by (I2)), and, for $x \in K_k$, $\mathrm{occ}(x) = \{j \in J_k : x \in V_j\}$, the *footprint* of x. Both are updated together with the chains; we do not spell out this bookkeeping.

Writing R and V for the matrices with columns R_j and V_j, (I1) and (I3) say that $R = B_k V$ with V invertible, and (I2) says that R is *reduced*. Unlike the decomposition used for persistence, V is not required to be triangular with respect to ord.

Insertion. INSERT(σ) for a k-simplex $\sigma \notin K$, $k \geq 1$, all of whose facets are in K:

1. give σ a key larger than all keys in use; create a slot j with $V_j = \{\sigma\}$ and $R_j = \partial\sigma$;
2. while $R_j \neq 0$ and $p = \mathrm{pivot}(\mathrm{low}(R_j))$ is defined: $R_j \leftarrow R_j + R_p$, $V_j \leftarrow V_j + V_p$;
3. if $R_j \neq 0$, set $\mathrm{pivot}(\mathrm{low}(R_j)) = j$.

The slots $p_1, \ldots, p_m$ added in step 2 form the *reduction chain* of σ. We call σ *positive* if $R_j = 0$ at the end and *negative* otherwise.

Deletion. DELETE(σ) for $\sigma \in K_k$, $k \geq 1$, without cofaces. Let $O = \mathrm{occ}(\sigma)$ and $Z = \{j \in O : R_j = 0\}$.

Case A ($Z \neq \emptyset$). Choose any $z \in Z$ (the implementations take one with $|V_z|$ minimal, ties broken by the smallest slot index). For every $j \in O \setminus \{z\}$: $V_j \leftarrow V_j + V_z$. Remove slot z.

Case B ($Z = \emptyset$). Let $p \in O$ be the slot whose $\mathrm{low}(R_p)$ has the smallest key. For every $j \in O \setminus \{p\}$: $R_j \leftarrow R_j + R_p$ and $V_j \leftarrow V_j + V_p$. Remove slot p and the entry $\mathrm{pivot}(\mathrm{low}(R_p))$.

Finally remove σ from K. Inserting or deleting a vertex changes only n_0. Figure 2 carries out both cases on a graph.

Theorem 1 (Exactness). *Let $k \geq 1$ and let $(R_j, V_j)_{j \in J_k}$ satisfy (I1)–(I3) for $B_k(K)$, for some injective key function.*

(a) $r_k = \#\{j \in J_k : R_j \neq 0\}$, and $\{V_j : R_j = 0\}$ is a basis of $Z_k(K)$.
(b) Let $\sigma \notin K$ be a k-simplex whose facets are all in K. The loop of INSERT(σ) *terminates, and afterwards the slots satisfy (I1)–(I3) for $B_k(K \cup \{\sigma\})$; every family satisfying (I1)–(I3) for $B_{k+1}(K)$ satisfies them, unchanged, for $B_{k+1}(K \cup \{\sigma\})$.*

(c) Let $\sigma \in K_k$ have no coface. Then $O \neq \emptyset$; after DELETE(σ)*, in Case A and in Case B, the slots satisfy (I1)–(I3) for $B_k(K \setminus \{\sigma\})$; every family satisfying (I1)–(I3) for $B_{k+1}(K)$ satisfies them, unchanged, for $B_{k+1}(K \setminus \{\sigma\})$.*

(d) Inserting a positive k-simplex increases β_k by one; inserting a negative one decreases β_{k-1} by one. A deletion in Case A decreases β_k by one; a deletion in Case B increases β_{k-1} by one. No other Betti number changes. Moreover, DELETE(σ) *is in Case A if and only if σ lies in the support of a k-cycle of K, and* INSERT(σ) *is positive if and only if σ lies in the support of a k-cycle of $K \cup \{\sigma\}$.*

Proof. (a) By (I1) and (I3), $R = B_k V$ with V invertible, so $\operatorname{rank} R = r_k$. The non-zero columns of R are linearly independent: in a non-empty sum of them, let R_i be the summand whose low ℓ has the largest key; every other summand has its low, hence all its cells, strictly below ℓ, so ℓ survives in the sum. Hence r_k is the number of non-zero R_j. Each V_j with $R_j = 0$ is a cycle; these chains are independent as part of a basis, and there are $n_k - r_k = \dim Z_k$ of them.

(b) Let $K' = K \cup \{\sigma\}$. Throughout the loop, $R_j = \partial V_j$ (true at the start and preserved by linearity because $R_p = \partial V_p$) and $V_j = \sigma + w$ with $w \in C_k(K)$. An addition is performed only when $\operatorname{low}(R_p) = \operatorname{low}(R_j) = \ell$; all cells of both chains have keys at most $\operatorname{ord}(\ell)$ and ℓ cancels, so afterwards $R_j = 0$ or $\operatorname{low}(R_j)$ is strictly smaller. Hence the loop terminates, after at most one addition per non-zero slot. On exit either $R_j = 0$ or $\operatorname{low}(R_j)$ is no pivot, so recording the pivot preserves (I2); (I1) holds. Since $C_k(K') = C_k(K) \oplus \mathbb{Z}_2\sigma$ and the old V_i form a basis of $C_k(K)$, the old chains together with $V_j = \sigma + w$ form a basis of $C_k(K')$: they are $n_k + 1$ chains spanning $\sigma = V_j + w$. For B_{k+1}: the chain group C_{k+1} is unchanged, each $R_i = \partial V_i$ is supported on facets of $(k+1)$-simplices of K', none of which is σ (Observation 2.1), and keys of existing cells are unchanged; so (I1)–(I3) continue to hold.

(c) Since the V_j span C_k, σ is a sum of some of them, so some V_j contains σ: $O \neq \emptyset$. Each operation $V_j \leftarrow V_j + V_q$ with $j \neq q$ is an invertible change of basis, so throughout the loop the family remains a basis of $C_k(K)$. For $j \in O \setminus \{q\}$ (with $q = z$ in Case A, $q = p$ in Case B), both V_j and V_q contain σ, so $V_j + V_q$ does not; slots outside O never contained σ. After the loop, V_q is therefore the only chain containing σ. Removing slot q leaves $n_k - 1$ independent chains, all in $C_k(K \setminus \{\sigma\})$, which has dimension $n_k - 1$: (I3) holds. *Case A.* Because $\partial V_z = R_z = 0$, the additions leave every R_j unchanged, so (I1) and (I2) hold; the removed slot had $R_z = 0$ and owned no pivot. *Case B.* Every R_j with $j \in O$ is non-zero, and by (I2) the lows are distinct, so p is well defined and $\operatorname{low}(R_p)$ is strictly smaller than $\operatorname{low}(R_j)$ for $j \in O \setminus \{p\}$. All cells of R_p have keys at most $\operatorname{ord}(\operatorname{low}(R_p)) < \operatorname{ord}(\operatorname{low}(R_j))$, so $\operatorname{low}(R_j) \notin R_p$ and $\operatorname{low}(R_j + R_p) = \operatorname{low}(R_j)$; in particular $R_j + R_p \neq 0$. Lows are unchanged and the low of R_p is released with slot p, so (I2) holds; (I1) holds by linearity. (The minimal choice of p is what guarantees this: adding an R_p with a larger low would move lows and require further reduction.) For B_{k+1}: σ has no coface, so no $R_i = \partial V_i$ of dimension $k+1$ contains σ and σ is not a pivot there; deleting the zero row σ leaves (I1)–(I3) intact.

(d) By (a), r_k increases by one exactly when the inserted σ is negative, and decreases by one exactly in Case B; n_k changes by ± 1; r_{k+1} is unchanged because only a zero row of B_{k+1} is added or removed (Observation 2.1); all other n_i and r_i are unchanged. Formula (1) gives the stated changes. If $z \in Z$, then V_z is a cycle containing σ. Conversely, if a cycle $c \in Z_k(K)$ contains σ, write c in the basis $\{V_j : R_j = 0\}$ of $Z_k(K)$ from (a); some V_j in this expansion contains σ, and that j lies in Z. For insertion, if σ is positive then the final V_j is a cycle of K' containing σ. Conversely, if a cycle c of K' contains σ, then $\partial\sigma = \partial(c + \sigma)$ lies in the column space of $B_k(K)$, so r_k does not increase, and by (a) the new slot has $R_j = 0$. □

Vertices need no slots: inserting a vertex increases n_0 and β_0 by one, and a deletable vertex is isolated, so deleting it decreases both by one (Observation 2.1 and (1)). A batch is processed one atomic edit at a time, so Theorem 1 makes the Betti numbers exact after every update. The choice of z in Case A is free; it only affects sparsity. Theorem 1 holds for every key function, so the order chosen in Section 4 is a matter of cost alone.

Initialisation and rebuilds. The DRB is initialised, and periodically rebuilt, by the static reduction with clearing [9], tracking V: for $k = d, d-1, \ldots, 1$, process the columns of B_k in increasing key order; if the k-simplex τ is the low of a non-zero reduced column R of B_{k+1}, *clear* it ($R_\tau := 0$, $V_\tau := R$); otherwise reduce it as in steps 2–3 of INSERT, starting from $V_\tau = \{\tau\}$, $R_\tau = \partial\tau$.

Proposition 1.1 (Rebuild)**.** *For every injective key function, the static reduction with clearing produces slots satisfying (I1)–(I3) in every dimension. Moreover, every V_τ consists of τ and cells with smaller keys. The same holds for the reduction without clearing. In particular the ranks it reports are exact even when V is not stored.*

Proof. We show $\operatorname{low}(V_\tau) = \tau$ for every column τ, by induction along the processing order. A non-cleared τ ends with $V_\tau = \{\tau\} + \sum V_p$ over pivot owners p processed earlier, hence with smaller keys, and by induction every cell of V_p has key at most $\operatorname{ord}(p)$. A cleared τ has $V_\tau = R$ with $\operatorname{low}(R) = \tau$. Ordering rows and columns by key, V is therefore upper unitriangular, hence invertible: (I3). (I1) holds for non-cleared columns by linearity, and for cleared ones because

$R = \partial V'$ for the slot V' of B_{k+1} that produced it (induction over dimensions), so $\partial V_\tau = \partial\partial V' = 0 = R_\tau$. (I2): a non-cleared column ends with $R_\tau = 0$ or with a low that no earlier column owns; cleared columns are zero. Without clearing every column is of the non-cleared kind and the same argument applies. The last statement follows from Theorem 1(a). □

Note that the correctness of clearing here follows from the invariants alone; we do not need the classical fact that a cleared column would have reduced to zero. After deletions, however, V is no longer triangular (Case A adds V_z, whose low may exceed that of V_j), so the DRB maintains ranks and a cycle basis but not a persistence pairing, and we claim none. This is the difference from vineyard updates [10], which keep V triangular for a changing order.

Rebuild policy. mmhm rebuilds with the Morse order of Section 4; it rebuilds after update t if $t - t_{\text{last}} \geq m$ or $\text{nnz}(R) + \text{nnz}(V) > (1+\alpha)\max(\text{nnz}_{\text{last}}, |K|)$, where nnz_{last} is the value right after the previous rebuild (defaults $m = \infty$, $\alpha = 1$), and the rebuild is charged to update t. drb-id uses insertion keys and never rebuilds; mmhm-norb uses the Morse order at the initial build only; drb-id-rb applies the same policy as mmhm but rebuilds in insertion order (Section 6.1).

4 The Morse order

Discrete Morse theory [21] enters only through the choice of keys at a rebuild. A rebuild computes an acyclic matching by collapses, in the spirit of [5], and orders cells by reverse removal time.

Definition 4.1 (Collapse matching and Morse order)**.** Run the following on $P := K$, with a counter $t := 0$ and a queue Q of the cells x that have exactly one immediate coface in P (*free faces*).

1. While $Q \neq \emptyset$: pop x; skip it unless $x \in P$ and x has exactly one immediate coface y in P. Then *pair* (x, y), set $\text{rm}(y) := t$, $\text{rm}(x) := t+1$, $t \leftarrow t+2$, remove y and x from P, and push every facet of y or x that has become free.
2. If $P \neq \emptyset$, choose a cell c of the highest dimension present in P (smallest vertex tuple), declare it *critical*, set $\text{rm}(c) := t$, $t \leftarrow t+1$, remove it, push the facets that have become free, and go to 1.

The *Morse order* is $\text{ord}(x) := -\text{rm}(x)$ (relabelled to $0, \ldots, |K|-1$). Write M for the set of pairs, Crit for the critical cells, Low for the cells x with $(x, y) \in M$ and Up for the cells y with $(x, y) \in M$; for $x \in \text{Low}$ let $\mu(x)$ be its partner. For a cell x, its *youngest facet* is the facet with the largest key and its *oldest coface* the coface with the smallest key.

The removed cells are always maximal in P, and P remains a subcomplex. For the paired cell y this uses the simplicial diamond property: if y had a coface $w \in P$, then w would have exactly two facets containing x, namely y and some $y' \neq y$, and $y' \in P$ because P is a subcomplex, so x would not be free. The queue discipline is not specified; all results below hold for every discipline. Figure 3 runs the procedure on a small complex.

A *gradient path* (V-path) is a sequence $\tau_0, \sigma_0, \tau_1, \sigma_1, \ldots, \tau_m$ with $(\tau_i, \sigma_i) \in M$ and τ_{i+1} a facet of σ_i different from τ_i.

Lemma 2 (Gradient pairs are apparent pairs)**.** *For every execution of the collapse procedure, with* $\text{ord} = -\text{rm}$*:*

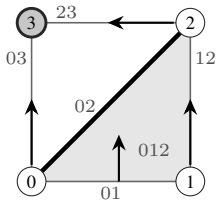
(a) ord *is a filtration order:* $\text{ord}(f) < \text{ord}(s)$ *whenever* f *is a facet of* s*.*
(b) rm *strictly increases along the lower cells of every gradient path* ($\text{rm}(\tau_{i+1}) > \text{rm}(\tau_i)$)*; in particular* M *has no closed gradient path, so it is an acyclic matching.*
(c) For every $(\tau, \sigma) \in M$*:* τ *is the youngest facet of* σ*, i.e.* $\tau = \text{low}(\partial\sigma)$*, and* σ *is the oldest coface of* τ*.*
(d) If $z \neq 0$ *is a cycle, then* $\text{low}(z) \notin \text{Up}$*.*
(e) In the static reduction under ord*, with or without clearing, for every* $(\tau, \sigma) \in M$ *the column* σ *is not cleared, receives no addition, and obtains pivot* τ*; thus* $R_\sigma = \partial\sigma$ *and* (τ, σ) *is an* apparent pair *[1].*

Proof. (a) Only maximal cells of P are removed (above), so every cell is removed before each of its facets.

(b) When σ_i is removed at time $\text{rm}(\sigma_i)$, its facet τ_{i+1} is still in P. Times $\text{rm}(\sigma_i)$ and $\text{rm}(\tau_i) = \text{rm}(\sigma_i) + 1$ are used by σ_i and τ_i, and $\tau_{i+1} \neq \tau_i$, so $\text{rm}(\tau_{i+1}) > \text{rm}(\tau_i)$. A closed gradient path would give $\text{rm}(\tau_0) < \text{rm}(\tau_0)$.

(c) The pair is removed at consecutive times, so $\text{ord}(\tau) = \text{ord}(\sigma) - 1$: the two cells are adjacent in the order. By (a) every facet of σ precedes σ, hence has key at most $\text{ord}(\tau)$, and every coface of τ follows τ, hence has key at least $\text{ord}(\sigma)$. (This is the elementary fact that a facet and a coface that are adjacent in a filtration order form an apparent pair.)

(a) Complex, matching, critical cells



$\beta = (1, 1, 0)$

(b) Collapse and Morse order

Collapse steps: pair $(01, 012)$, pair $(1, 12)$; no free face is left, so 02 is critical; pair $(2, 23)$, pair $(0, 03)$; 3 is critical.

cell	**3**	0	03	2	23	**02**	1	12	01	012
	crit.	pair		pair		crit.	pair		pair	
key ord	0	1	2	3	4	5	6	7	8	9
removal time rm	9	8	7	6	5	4	3	2	1	0

(c) Boundary matrices in the Morse order

$B_2 =$

	012
03	0
23	0
02	1
12	1
01	$\boxed{1}$

$B_1 =$

	03	23	02	12	01
3	1	1	0	0	0
0	$\boxed{1}$	0	1	0	1
2	0	$\boxed{1}$	1	1	0
1	0	0	0	$\boxed{1}$	1

Boxed: the apparent pivots, $R_y = \partial y$ with low x for $(x, y) \in M$. Column 01 (grey) is cleared, since 01 is the low of R_{012}. Only the shaded critical column 02 is reduced (Corollary 2.1).

(d) Reducing the critical column 02

	$\partial 02$	$\xrightarrow{\mu(2)=23}$	$+\partial 23$	$\xrightarrow{\mu(0)=03}$	$+\partial 03$
3	0		1		0
0	1		$\boxed{1}$		0
2	$\boxed{1}$		0		0
1	0		0		0

Each addition replaces the low, a lower cell, by the other facet of its partner, following the gradient paths $2 \to 23 \to 3$ and $0 \to 03 \to 3$ (Corollary 2.2). The column ends at $\partial^M 02 = 3 + 3 = 0$: $V_{02} = 02 + 23 + 03$ is the cycle around the empty triangle, so 02 is positive.

Figure 3: The Morse order on a small complex K: the square 0123 with diagonal 02 and the triangle 012 (shaded); the triangle 023 is not in K. (a) Arrows point from x to $\mu(x)$ for the gradient pairs found by the collapse; the critical cells 3 and 02 are drawn heavy. (b) Reading the removal times from right to left replays the collapse; the keys are $\mathrm{ord} = 9 - \mathrm{rm}$. Each pair is adjacent in the order and its lower cell is the youngest facet of its partner, so each pair is apparent (Lemma 2). (c)–(d) Rows and columns are in key order, rows from top to bottom, so a low is the lowest non-zero entry (boxed). Every entry is recomputed and checked by `reference/figure_examples.py`.

(d) Let $u = \mathrm{low}(z) \in \mathrm{Up}$ with $u = \mu(w)$. The coefficient of w in $\partial z = 0$ vanishes and $w \in \partial u$, so some $u' \in z$ with $u' \neq u$ has w as a facet. By (c), u is the oldest coface of w, so $\mathrm{ord}(u') > \mathrm{ord}(u)$, contradicting $u = \mathrm{low}(z)$.

(e) If σ were cleared, it would be the low of a reduced column R of the next dimension; $R = \partial V'$ (Proposition 1.1) is a non-zero cycle with low $\sigma \in \mathrm{Up}$, contradicting (d). Columns are processed in increasing key order. When σ is processed, every earlier column x has $R_x = 0$ (if cleared) or $R_x = \partial V_x$ with all cells of V_x of key at most $\mathrm{ord}(x) < \mathrm{ord}(\sigma)$ (Proposition 1.1); by (c) no such cell has τ as a facet, so $\tau \notin R_x$ and $\mathrm{pivot}(\tau)$ is undefined. Since $\mathrm{low}(\partial\sigma) = \tau$ by (c), the reduction loop performs no addition and assigns $\mathrm{pivot}(\tau) = \sigma$. □

Corollary 2.1 (Work happens only on critical columns)**.** *In the static reduction with clearing under the Morse order, the column of every $x \in \mathrm{Low}$ with $\dim x \geq 1$ is cleared, the column of every $y \in \mathrm{Up}$ is apparent, and therefore every column addition is performed while reducing the column of a critical cell.*

Proof. For $(x, y) \in M$ with $\dim y = k + 1 \geq 2$, Lemma 2(e) applied to B_{k+1}, which is processed before B_k, gives $R_y = \partial y$ with $\mathrm{low}(R_y) = x$, so x is cleared. Upper cells are apparent by Lemma 2(e). Every remaining column belongs to a critical cell. □

Morse boundary. By Lemma 2(b) the following recursion is well founded and defines a linear map $\Pi\colon C_{k-1} \to C_{k-1}$: $\Pi(\tau) = \tau$ for $\tau \in \mathrm{Crit}$, $\Pi(\tau) = 0$ for $\tau \in \mathrm{Up}$, and $\Pi(\tau) = \sum_{\tau' \in \partial\mu(\tau),\, \tau' \neq \tau} \Pi(\tau')$ for $\tau \in \mathrm{Low}$. Thus $\Pi(\tau)$ is the sum, over all gradient paths from τ to a critical cell, of their endpoints, counted mod 2. The *Morse boundary* of a critical k-cell c is $\partial^M c := \Pi(\partial c)$, the boundary of the Morse complex of the matching [21]; the Morse boundary matrices B_k^M have rows $\mathrm{Crit} \cap K_{k-1}$ and columns $\mathrm{Crit} \cap K_k$. For $y \in \mathrm{Up}$ with partner x, the definition of $\Pi(x)$ gives

$$\Pi(\partial y) = \Pi(x) + \sum_{f \in \partial y,\, f \neq x} \Pi(f) = 0. \tag{2}$$

Corollary 2.2 (Critical columns perform the Morse reduction)**.** *Consider the static reduction with clearing under the Morse order and, next to it, the standard reduction with clearing of the Morse boundary matrices $B_d^M, \ldots, B_1^M$, critical cells taken in increasing key order. For every critical k-cell c, $k \geq 1$:*

(i) the additions performed on column c are first additions of apparent columns $R_y = \partial y$ ($y \in \mathrm{Up}$), then additions of columns of critical cells;

(ii) if c is not cleared, then when the first phase ends the column contains no cell of Low *and its restriction to* Crit *equals $\partial^M c$;*

(iii) the second phase performs, one for one, the additions that the Morse reduction performs on column c; c is cleared exactly when it is cleared in the Morse reduction; and the final column R_c contains no cell of Low*, while $R_c \cap \mathrm{Crit}$ is the reduced Morse column of c.*

Each first-phase addition $R \leftarrow R + \partial y$ with $\mathrm{low}(R) = x$ and $y = \mu(x)$ replaces x by the other facets of y; it is one step of the discrete gradient flow, and $\Pi(R)$ does not change by (2). This is the precise sense in which the reduced column of a critical cell "traces its gradient flow" (Figure 3d). R_c may still contain cells of Up; we observed this in the randomised checks and claim nothing about them.

Proof. We use the rule of Definition 4.1 that a critical cell is chosen of the highest dimension present in P. (Lemma 2 did not use it.)

Claim (separation). For every q, every critical q-cell has a smaller key than every q-cell of Low. Indeed, when a critical q-cell t is removed, P contains no $(q+1)$-cell. If $\ell \in \mathrm{Low}$ were a q-cell with $\mathrm{rm}(\ell) > \mathrm{rm}(t)$, its partner $\mu(\ell)$, a $(q+1)$-cell removed at time $\mathrm{rm}(\ell) - 1 \geq \mathrm{rm}(t)$, would be in P at that moment.

We argue by induction over the dimensions (top-down) and, within B_k, over the columns in increasing key order. The statement about clearing is shown below; a cleared column receives no addition and has $R_c = 0$, matching the zero column of the Morse reduction. So let c be a column that is not cleared, and let R denote its column during its reduction; initially $R = \partial c$.

First phase. Suppose $\ell = \mathrm{low}(R) \in \mathrm{Low}$ and let $y = \mu(\ell)$. All cells added to V_c so far are c and upper cells with smaller keys, so ℓ is a facet of a cell x with $\mathrm{ord}(x) \leq \mathrm{ord}(c)$; since y is the oldest coface of ℓ (Lemma 2(c)) and $y \neq c$, $\mathrm{ord}(y) < \mathrm{ord}(c)$. By Lemma 2(e), the column of y has been processed, holds $R_y = \partial y$ and owns $\mathrm{pivot}(\ell)$; the algorithm adds it, which strictly decreases the low and preserves $\Pi(R)$ by (2). The phase ends with $R = 0$ or with $t = \mathrm{low}(R) \notin \mathrm{Low}$. In the latter case R is a non-zero cycle (by (I1), $R = \partial V_c$), so $t \notin \mathrm{Up}$ by Lemma 2(d), and $t \in \mathrm{Crit}$. By the separation claim every cell of R, having key at most $\mathrm{ord}(t)$, lies outside Low. As Π fixes critical cells and kills upper cells, $R \cap \mathrm{Crit} = \Pi(R) = \Pi(\partial c) = \partial^M c$. This proves (ii) and the first part of (i).

Clearing. The cells cleared in B_k are the lows of the non-zero reduced columns of B_{k+1}: the lower cells (lows of apparent columns) and the lows of non-zero critical columns. By the induction hypothesis (iii) for dimension $k+1$, the latter are critical and coincide with the lows of the non-zero reduced Morse columns, i.e. with the cells cleared in the Morse reduction. So a critical c is cleared in one reduction if and only if in the other; in the top dimension nothing is cleared in either.

Second phase. A pivot on a critical row cannot be owned by an apparent column (its low is in Low) or a cleared one (zero), so it is owned by a critical column c' processed earlier. By the induction hypothesis $R_{c'}$ has no cell of Low, it is a cycle, its low is critical and equals $\mathrm{low}(R_{c'} \cap \mathrm{Crit})$, and $R_{c'} \cap \mathrm{Crit}$ is the reduced Morse column of c'; hence the pivot tables of the two reductions agree on critical rows. Adding such columns keeps $R \cap \mathrm{Low} = \emptyset$ and keeps R a cycle, so while $R \neq 0$ its low is critical and equals $\mathrm{low}(R \cap \mathrm{Crit})$. The sequence of pivot look-ups and additions on $R \cap \mathrm{Crit}$, which starts from $\partial^M c$, is therefore exactly the Morse reduction of column c, and no apparent column is added any more. This proves (i) and (iii). □

Remark 4.2 (Relation to known results). Lemma 2 is not new in substance. Its part (c) is the elementary fact noted in its proof, and the relation between gradient pairs and apparent pairs is known in both directions: the apparent pairs of any simplexwise filtration form a discrete gradient, which Ripser uses to skip columns [1], and Bauer and Roll [2] relate the gradient pairs of a (generalised) discrete Morse function to the zero-persistence apparent pairs of a lexicographically refined filtration order. Lemma 2 is a self-contained version for the order produced by our collapse procedure, in the form used below: for every queue discipline, with and without clearing, and with part (d), on which Corollary 2.2 rests. Mischaikow and Nanda [32] showed that Morse reductions compatible with a filtration preserve persistent homology; Corollary 2.2 is a matrix-level form of this for our order: on critical rows and columns, the static reduction *is* the reduction of the Morse complex, and by Corollary 2.1 the other columns cost no additions. This is what the full-morse baseline exploits. In the language of algebraic Morse theory [26, 38], the first phase of Corollary 2.2 is Gaussian elimination of the invertible entries $\langle \partial y, x \rangle = 1$ of the matched pairs, which is how the Morse complex arises algebraically; the order makes the reduction algorithm perform these eliminations by itself, without a separate Morse-reduction pass.

Remark 4.3 (Scope). Lemma 2 and its corollaries describe the state produced by a rebuild. Between rebuilds the matching is not maintained, inserted cells get keys larger than all others, and deletions add chains whose lows may exceed those of their targets. We make no claim that the DRB stays sparse between rebuilds; the Morse order is a heuristic for small footprints and short reduction chains, its effect is measured, and the rebuild policy bounds the drift of $\mathrm{nnz}(R) + \mathrm{nnz}(V)$.

5 Update cost and the limits of locality

5.1 Output-sensitive cost

We state the cost in two storage models. In the *ordered-set model*, a chain is a balanced search tree keyed by ord and a footprint is a set of slots; inserting, removing or testing one cell or slot, one pivot-table operation and one low query are *elementary operations*, each of $O(\log n)$ time, $n = |K|$. In the *sorted-array model*, which is what the engine implements (Section 6.1), a chain is an increasing array of keys, its low is its last entry, and a sum $x + y$ of two chains is a linear merge that costs $\Theta(|x| + |y|)$, the size of the target included. For an insertion we write $R^{(i)}$ and $V^{(i)}$ for the working column just before the i-th addition of its reduction chain.

Proposition 3 (Output-sensitive cost)**.** *Let σ be a k-simplex, $k \geq 1$.*

(a) INSERT(σ) *with reduction chain $p_1, \dots, p_m$ costs $\Theta\big(k + 1 + \sum_{i=1}^{m}(1 + |R_{p_i}| + |V_{p_i}|)\big)$ elementary operations in the ordered-set model and $\Theta\big(k + 1 + \sum_{i=1}^{m}(1 + |R^{(i)}| + |V^{(i)}| + |R_{p_i}| + |V_{p_i}|)\big)$ time in the sorted-array model. In both, the p_i are pairwise distinct, so $m \leq r_k$ and $\sum_i(|R_{p_i}| + |V_{p_i}|) \leq \mathrm{nnz}(R) + \mathrm{nnz}(V)$ of the DRB of B_k.*

(b) DELETE(σ) *with footprint $O = \mathrm{occ}(\sigma)$ costs, including the maintenance of the footprints and of* pivot*, $\Theta\big(|O|\,(1 + |V_z|)\big)$ elementary operations in Case A and $\Theta\big(|O|\,(1 + |R_p| + |V_p|)\big)$ in Case B in the ordered-set model, and $\Theta\big(\sum_{j\in O}(1 + |V_j| + |V_z|)\big)$ in Case A and $\Theta\big(\sum_{j\in O}(1 + |R_j| + |V_j| + |R_p| + |V_p|)\big)$ in Case B in the sorted-array model.*

(c) An update costs the sum over its atomic edits; a vertex edit costs $O(1)$.

Proof. (a) Each addition touches every cell of R_p and V_p once, and in the sorted-array model also every cell of the working column, and performs one pivot look-up and one low query; the final chain V_j, of size at most $1 + \sum_i |V_{p_i}|$, is entered into the footprints; $|\partial\sigma| = k + 1$. Along the chain the lows strictly decrease (proof of Theorem 1(b)), and each slot owns one low, so no slot is added twice. (b) Scanning O to determine Z, and z or p, costs $O(|O|)$ (sizes $|V_j|$ are stored). In Case A each of the $|O| - 1$ additions touches the $|V_z|$ cells of V_z in V_j and in the footprints (in the sorted-array model also the $|V_j|$ cells of the target), and removing slot z removes it from $|V_z|$ footprints. Case B is the same with R_p and V_p, plus one pivot removal. □

The engine is a hybrid: when short chains are added to a long working column it switches the column to a max-heap, which brings an insertion to the ordered-set bound up to a logarithmic factor, whereas its deletions pay the sorted-array cost (Section 6.1). The engine does not log these costs. Per update it records the number of column additions (the number m of terms in (a), and $|O| - 1$ per deletion in (b)), the sum of the footprints $|\mathrm{occ}(\sigma)|$ of the deleted simplices, and $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ (Section 6.1). These are proxies: they count the terms of the sums above but not the sizes of the chains involved, so they need not move in the same direction as running time. Neither model counts the engine's lazy footprint lists: entries left behind by earlier eliminations are skipped when $\mathrm{occ}(\sigma)$ is read, which adds work proportional to the stale entries met.

The quantities in Proposition 3 — footprints $|\mathrm{occ}(\sigma)|$, reduction-chain lengths and the sizes of the chains involved — depend on the whole history and on the order, not on s_t. We report proxies for them as measurements and state no locality theorem. Proposition 4 shows that none is possible for this data structure.

5.2 No worst-case locality

Proposition 4 (No worst-case locality for the DRB)**.** *Let K be either (W5a) the cycle graph C_L, $L \geq 3$, and σ one of its edges, with $n := L$; or (W5b) a triangulated 2-sphere (more generally, a connected closed triangulated surface) and σ one of its triangles, with $n := n_2$. Let $k = \dim\sigma$. Then, for every key function and every choice in Case A:*

(a) for every DRB of $B_k(K)$ satisfying (I1)–(I3), DELETE(σ) *is in Case A with $|V_z| = n$ and performs at least n elementary operations;*

(b) for every DRB of $B_k(K \setminus \{\sigma\})$ satisfying (I1)–(I3), INSERT(σ) *is positive, ends with $V_j = K_k$, and its reduction chain satisfies $\sum_i |V_{p_i}| \geq n - 1$;*

(c) if S is any DRB state of $K \setminus \{\sigma\}$ and S' is the state after INSERT(σ)*, then* DELETE(σ) *applied to S' restores S exactly. Hence, after the first deletion, toggling σ alternates between two fixed states S_0, S_1, and each toggle costs at least n and, in the ordered-set model, at most $O\big(n + \mathrm{nnz}(R) + \mathrm{nnz}(V)\big)$ elementary operations, with* nnz *taken in S_0.*

So on these families every toggle of σ, an update of size $s_t = 1$, costs this algorithm $\Omega(n)$ work in either storage model, and $\Theta(n)$ elementary operations whenever $\mathrm{nnz}(R) + \mathrm{nnz}(V) = O(n)$ in S_0. Other updates of size one, such as inserting a pendant edge, can of course be cheap.

The statement bounds the work of *this* algorithm, which must maintain (I1)–(I3) and the footprint index; it is not a lower bound for computing Betti numbers under edits. For (W5a), $\beta_1 = n_1 - n_0 + \beta_0$ and β_0 can be maintained by dynamic connectivity in polylogarithmic amortised time per edge edit [25]; for (W5b), see Remark 5.1. What the proposition does show is that any method that stores cycle representatives explicitly, as (I3) forces, has to write the n-cell cycle when it appears.

Proof. In both cases $Z_k(K) = \{0, K_k\}$. For C_L, a 1-cycle is a set of edges meeting every vertex an even number of times, and the only non-empty such set is the whole cycle. For a connected closed surface, a 2-cycle that contains a triangle contains, for each edge of it, the other triangle on that edge; as the surface is strongly connected, it contains every triangle. Also $Z_k(K \setminus \{\sigma\}) = \{0\}$, since the only non-zero cycle of K contains σ.

(a) By Theorem 1(a) exactly one slot z has $R_z = 0$, and $V_z = K_k \ni \sigma$. So $z \in O$, the deletion is in Case A with $Z = \{z\}$, and removing slot z removes it from the footprints of all n cells of V_z.

(b) $\beta_k(K) = \beta_k(K \setminus \{\sigma\}) + 1$, so by Theorem 1(d) σ is positive; the final V_j is then a non-zero cycle of K, i.e. $V_j = K_k$. Since $V_j = \{\sigma\} + \sum_i V_{p_i}$, we get $n \leq 1 + \sum_i |V_{p_i}|$.

(c) In S no chain contains σ, and INSERT(σ) creates one slot j and modifies no other. So in S' we have $\mathrm{occ}(\sigma) = \{j\}$ and $R_j = 0$ by (b): DELETE(σ) is in Case A with $O = Z = \{j\}$, modifies nothing and removes j. The key given to σ differs between insertions, but it is always the largest key in use and the algorithms only compare keys, so every insertion from S_0 performs the same computation. The bounds follow from (a), (b) and Proposition 3. □

Remark 5.1 (Surfaces need no linear algebra). Let S be a connected triangulated surface, possibly with boundary (every edge lies in one or two triangles and every vertex link is a path or a cycle). Over $\mathbb{Z}_2$: $\beta_0 = 1$; $\beta_2 = 1$ if no edge lies in exactly one triangle and $\beta_2 = 0$ otherwise (by strong connectivity a non-zero 2-cycle is the set of all triangles, whose boundary is the set of boundary edges); and $\beta_1 = \beta_0 + \beta_2 - \chi(S)$ with $\chi = n_0 - n_1 + n_2$. Under atomic edits that keep S a connected surface, n_0, n_1, n_2 and the number of boundary edges are maintained in $O(1)$ per edit from per-edge triangle counts. All Betti numbers are then available in $O(1)$ per edit without any matrix — provided that connectivity and the surface property are known in advance; the counters do not certify connectivity. This is the euler method. Consequently, workloads consisting of connected surfaces with known connectivity cannot evaluate the algebraic part of a homology-maintenance method: a method that switches to these formulas returns correct Betti numbers there whether or not its matrix updates are correct.

6 Experimental evaluation

The experiments support four claims. The dynamic reduced basis (DRB) is exact in practice as in theory (Section 6.2). On workloads whose edits create and destroy only small cycles, maintaining it is much cheaper than recomputing the Betti numbers (Section 6.3). Rebuilding and the Morse order add further gains under conditions that the ablations identify (Section 6.4.1). The cost of the representation is not local in general, as Proposition 4 predicts (Section 6.4.2). Detailed analyses, all result tables and the sensitivity study are in the supplementary material (numbered S1, S2, …); every number, table and figure is regenerated by scripts from the raw per-update records (`experiments/README.md`).

6.1 Setup

Workloads. Five deterministic generators produce sequences of updates, each a batch of atomic edits that keeps the complex simplicial (Table 1). For W1, W2, W3 and W5 the Betti numbers of every state are known in closed form. W4 is a flag complex truncated at dimension D, so the boundary of every $(D+2)$-clique is a D-cycle that is never filled; β_D/n is 2.8 to 6.2 at the start. W4 therefore exercises the algebra on non-manifold complexes in which every Betti number changes; we draw no conclusion about the topology of point clouds from it. Generator details are in Section S1.

Table 1: Workloads. s_t: atomic edits per update; c: number of cavities; h: number of holes. Every timed configuration ran with 3 seeds and $T = 2{,}000$ updates.

	Complex and sweep	Update	Betti numbers	max. $\lvert K_0 \rvert$
W1	icosphere, levels $L = 4, \ldots, 7$; torus $n \times n$, $n = 64, \ldots, 500$	open or close a disk-shaped hole; $s_t \leq 61$	closed form; β_1, β_2 change with h	1,500,000
W2	Freudenthal N^3 grid, $N = 7, \ldots, 33$; $s = 1, \ldots, 64$	s stellar $1 \to 4$ moves or their inverses; $s_t = 16s$	$(1, 0, 0, 0)$ after every update	955,059
W3	as W2, $N = 14, 20, 27$	carve or refill a ball of tetrahedra	$(1, 0, c, 0)$	525,043
W4	flag complex of n moving points, mean degree 10; $n = 1000, \ldots, 16000$, $D \in \{2, 3\}$	move one point: delete and reinsert its star	all vary; no closed form	254,196
W5	(a) cycle C_L, $L = 10^3, \ldots, 10^6$; (b) icosphere, levels $3, \ldots, 8$	toggle one edge (a) or one triangle (b)	closed form, alternating	3,932,162

Table 2: Methods. *Canonical* order: by dimension, then lexicographically by vertex tuple. Two further methods appear only in the supplement: static recomputation in a fresh Morse order after every update, and $O(1)$ surface counters.

Method	Order	Rebuilds	Role
full-id	insertion	every update	static recomputation, ranks only
full-sorted	canonical	every update	static recomputation, history-independent order
gudhi	GUDHI's own	every update	external reference: GUDHI simplex tree, persistence over $\mathbb{Z}_2$
drb-id	insertion	never	DRB alone
mmhm-norb	Morse of K_0	never	DRB + Morse order
drb-id-rb	insertion	policy	DRB + rebuilds
mmhm	Morse	policy	DRB + Morse order + rebuilds (Sections 3 and 4)
drb-rand	random	never	DRB in an unstructured order (ablation)

Methods. Table 2 lists the methods; all except gudhi run in one C++ engine and share its complex store, parser and reduction code. The four DRB variants separate the mechanisms: drb-id is the DRB alone, in insertion order; mmhm-norb adds the Morse order of K_0; drb-id-rb adds rebuilds in insertion order; mmhm has both. Rebuilds follow the policy of Section 3 with $m = \infty$ and $\alpha = 1$. The static baselines recompute all ranks after every update by the reduction with clearing. full-id uses the insertion order, which depends on the history of edits and whose cost drifts upward on W3 and W4; full-sorted uses a canonical order that depends only on the current complex. On each instance we take the faster of the two as the *static reference*. gudhi computes persistence through GUDHI's Python interface; it is a reference point for a widely used library, not the best possible static recomputation, and not our primary evidence. We did not benchmark implementations of zigzag persistence or of warm-started reductions.

Platform and timing. All runs use one cloud virtual machine (Intel Xeon at 2.10 GHz, 4 cores, 15 GiB, Ubuntu 24.04, g++ 13.3 with `-O2`), single-threaded, with at most two timed runs at a time. The engine stores a chain as a sorted array of keys, so $\mathbb{Z}_2$ additions are linear merges (Section 5.1). The time of an update is the wall-clock time of processing its batch, including any rebuild; parsing, verification and the initial build are excluded. gudhi processes only the first $T' = 200$ updates on instances above 300,000 simplices. For each instance and method we pool the per-update latencies of the three runs and report the *median*, the *p99* and the *amortised latency*, the total time of all updates divided by their number. A *speedup* is a ratio of amortised latencies over the same updates; for gudhi on a prefix, both methods are taken over that prefix. Slopes are least-squares fits of log latency against log size. A difference is *within noise* when the per-seed ranges overlap; on W1–W4 the seeds are different instances, so this is a weak criterion. Repeated identical runs on W2 varied by 4.5–8.4 % (coefficient of variation of the amortised latency), and by up to 28.9 % on microsecond-scale updates (Section S2). Besides time, every run logs per update the column additions, the footprints of the deletions and $\mathrm{nnz}(R) + \mathrm{nnz}(V)$; these are proxies for the terms of Proposition 3.

Correctness checks. Every exact run records its Betti numbers after every update. They are compared with three references: the engine's full-id (on every state of every instance), the closed forms (W1, W2, W3, W5), and GUDHI (all updates up to 300,000 simplices, the first T' above). The engine also checks itself against a recomputation every 50 updates, its randomised tests compare every exact method with a brute-force rank computation, and a pure-Python implementation of the DRB that shares no code with the engine is cross-checked on 8 committed samples.

Table 3: Coverage of the correctness checks: Betti vectors reported by exact runs and the fractions compared with at least one, at least two and all three references (full-id, closed form, GUDHI), and with at least one reference outside the engine. W4 has no closed form.

workloads	vectors	≥ 1 ref.	≥ 2 ref.	all 3	≥ 1 external
W5 witnesses	482,652	482,652 (100.0%)	461,052 (95.5%)	250,974 (52.0%)	482,652 (100.0%)
W4 flag complexes	558,279	558,279 (100.0%)	474,237 (84.9%)	0 (0.0%)	516,258 (92.5%)
W1 surfaces	422,019	422,019 (100.0%)	405,819 (96.2%)	228,771 (54.2%)	422,019 (100.0%)
W3 cavities	360,783	360,783 (100.0%)	355,383 (98.5%)	297,765 (82.5%)	360,783 (100.0%)
W2 refine/coarsen	708,957	708,957 (100.0%)	703,557 (99.2%)	549,891 (77.6%)	708,957 (100.0%)
all exact runs	2,532,690	2,532,690 (100.0%)	2,400,048 (94.8%)	1,327,401 (52.4%)	2,490,669 (98.3%)
Python reference (`reference/`): 8 committed samples only, 3 methods, 2,514 vectors, 0 mismatches					

Table 4: Summary at one representative instance per family (three runs pooled). Amortised and p99 latency in ms. Static reference: the faster of full-id (^{i}) and full-sorted (^{s}) on that instance. Speedup: amortised latency of the static reference over that of the method, over the same updates; $^{\circ}$: within noise. Bold p99: above the static reference beyond noise. W1 and W5 are discussed in Section 6.4.2. All instances: Tables S10 and S12.

		static reference		drb-id		mmhm		speedup	
instance	$\lvert K_0\rvert$	amort.	p99	amort.	p99	amort.	p99	drb-id	mmhm
W2 $N = 33$, $s = 8$	955,059	14.6^{s}	21.9	0.165	0.404	0.165	0.333	88.7	88.6
W3 $N = 27$	525,043	10.8^{s}	20.1	4.60	**29.6**	1.81	7.40	2.36	5.99
W4 2D $n = 16$k, $D = 2$	254,196	12.0^{i}	19.4	0.169	0.644	0.127	0.402	71.0	94.5
W1 sphere $L = 7$	983,042	30.7^{i}	52.2	37.1	**305**	16.6	**151**	0.827	1.85
W1 torus 500×500	1,500,000	38.2^{i}	79.0	45.1	**313**	34.0	**397**	0.848	1.13
W5a $L = 1{,}000{,}000$	2,000,000	36.3^{i}	48.1	14.2	**154**	20.1	**101**	2.55	1.81
W5b level 8	3,932,160	178^{i}	240	21.5	203	28.7	**270**	8.27	6.20

6.2 Correctness and validation

The 1,290 runs of exact methods reported 2,532,690 Betti vectors. Every vector was compared with at least one reference, 98.3 % with a reference outside the engine and 52.4 % with all three (Table 3), and none disagreed (0 mismatches); the Python reference agreed on all 2,514 vectors it checked. This validation is empirical: it covers our implementation on our workloads, which include changes of every Betti number, non-manifold complexes and both deletion cases. Exactness in general rests on Theorem 1. Per-method counts are in Table S3.

6.3 Performance and scalability

Figure 4 shows how the median latency scales on W2, W3 and W4, and Table 4 summarises one representative instance per family, with the p99 next to the amortised latency.

W2: localised updates, constant topology. On W2 the Betti numbers are $(1, 0, 0, 0)$ after every update, but each update consists of $16s$ atomic edits, and every edit changes one Betti number (Theorem 1(d)). The DRB latency depends only weakly on the size of the complex: over five sizes the median grows with slope 0.24 to 0.27, against 1.00 and 1.04 for the static baselines (Figure 4(a)). The speedup over the static reference therefore grows with the complex, from 2.29–2.89 at 10,739 simplices to 88.6–89.9 at 955,059. All four DRB variants are within noise of each other here, and no rebuild fired: the gain is that of the dynamic basis itself, not of the Morse order or of rebuilds. The DRB cost grows linearly with the edit size (slopes 0.90 to 0.88 in s; Figure 4(b)), whereas recomputation hardly depends on it; at $s = 64$, that is 1024 atomic edits per update, the DRB variants are still 4.80–5.85 times faster than full-id; larger batches were not tested. Against gudhi on the same prefix, mmhm is 948 times faster at the largest size, a ratio that also reflects GUDHI's richer output and its interface. Because the topology of W2 does not change from update to update, W2 alone does not show the behaviour under topology changes; W3 and W4 do.

W3: changing topology and a growing basis. On W3 every carving creates a cavity and every refill destroys one. Here the maintained basis grows: without rebuilds, $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ rises over a run at $N = 20$ from 812,580 to 4,505,833 (drb-id), and the latency rises with it (Figure 5(b)). Rebuilds triggered by this growth (6.0 per run for mmhm) reset the basis and keep the latency from drifting upward. mmhm is the fastest method at every W3 size, 1.45 to 5.99 times faster than the static reference, which is full-sorted here; at $N = 27$ it is 7.45 and 8.44 times faster over the first

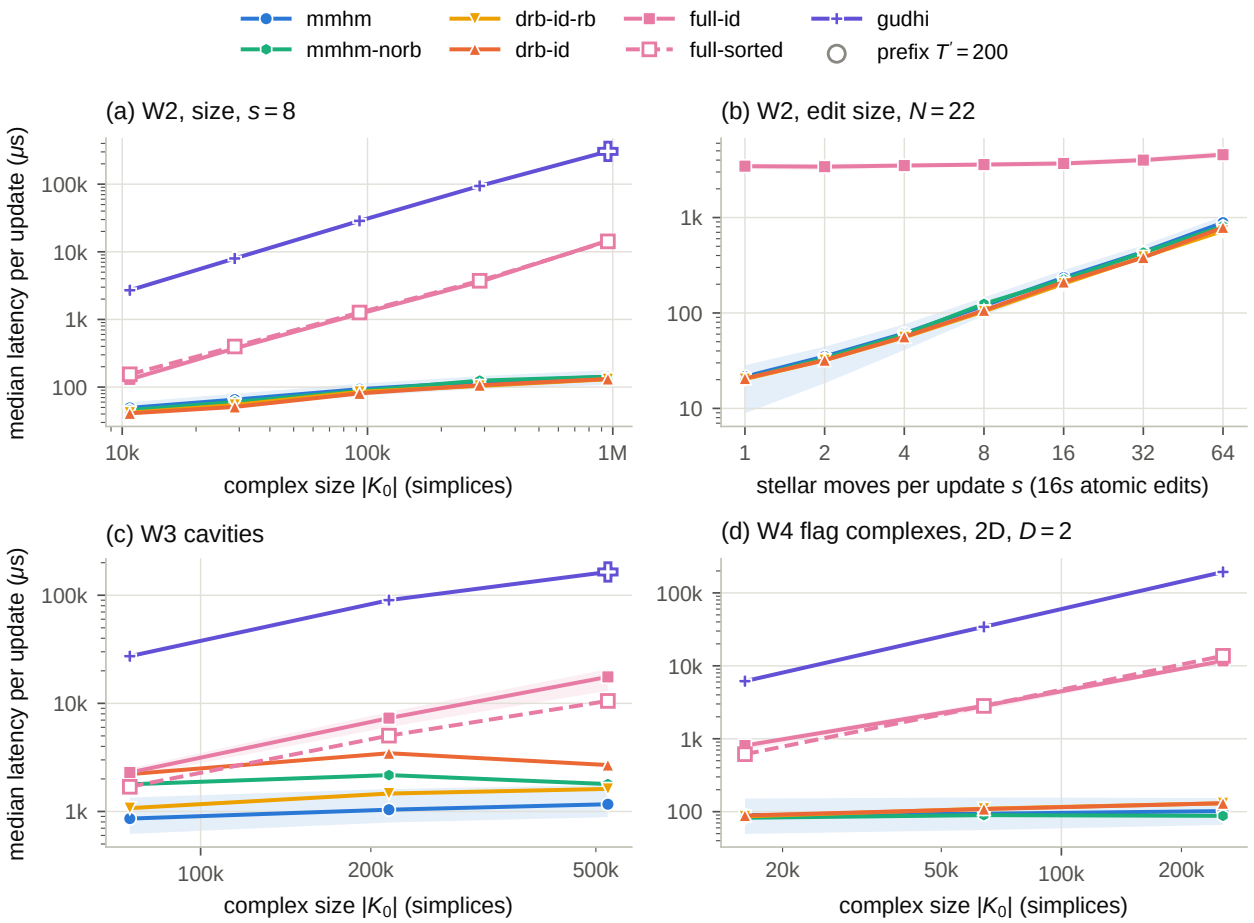


Figure 4: Median per-update latency (log–log; three runs pooled; bands: interquartile range of mmhm and full-id). (a) W2 against $|K_0|$ at $s = 8$; (b) W2 against the edit size s at $N = 22$ (only the methods run at all seven values of s); (c) W3; (d) W4 in the square with $D = 2$. Hollow markers: gudhi on the prefix T'. Static recomputation in the Morse order and the W1 surfaces: Figure S1.

and the last 200 updates, and its p99 is lower as well (Table 4). The variants without rebuilds are slower than full-sorted at the smallest size and have a higher p99 there; Section 6.4.1 separates the contributions of rebuilding and of the order.

W4: non-manifold complexes in which every Betti number changes. The DRB variants are faster than the static reference on every W4 instance, 3.25 to 6.09 times at $n = 1000$ and 12.2 to 109 times at $n \geq 4000$, and their median latency hardly depends on the size (slopes 0.02 to 0.15 against 0.97 for full-id, three sizes; Figure 4(d)). Their p99 is below that of the static reference except on the two instances with $n = 1000$ and $D = 3$, where the difference is within noise. No rebuild fired under the default policy, and on the larger instances the Morse order makes the DRB faster (Section 6.4.1).

What an update costs. Proposition 3 expresses the cost of an edit through its column additions and the footprints of its deletions, each weighted by the sizes of the chains involved; the engine logs the two counts but not the chain sizes (Figure 6). A W2 update, with its 128 atomic edits, performs a median of 156 to 166 column additions depending on the variant, and a W4 update 204 to 276; a W3 update, which creates or destroys a cavity, performs 4,200 to 5,910. The number of additions orders the updates by latency well on W4 (Spearman's ρ from 0.90 to 0.95 across the variants) and less well on W2 (0.63 to 0.85) and W3 (0.46 to 0.53). On W3, at an equal number of additions, the variants that rebuild, whose basis is smaller (Figure 5(b)), are faster than their counterparts without rebuilds; this is consistent with Proposition 3, in which an addition costs time proportional to the sizes of the chains involved, but we did not log those sizes. The counts compare variants; they do not predict the time of an update.

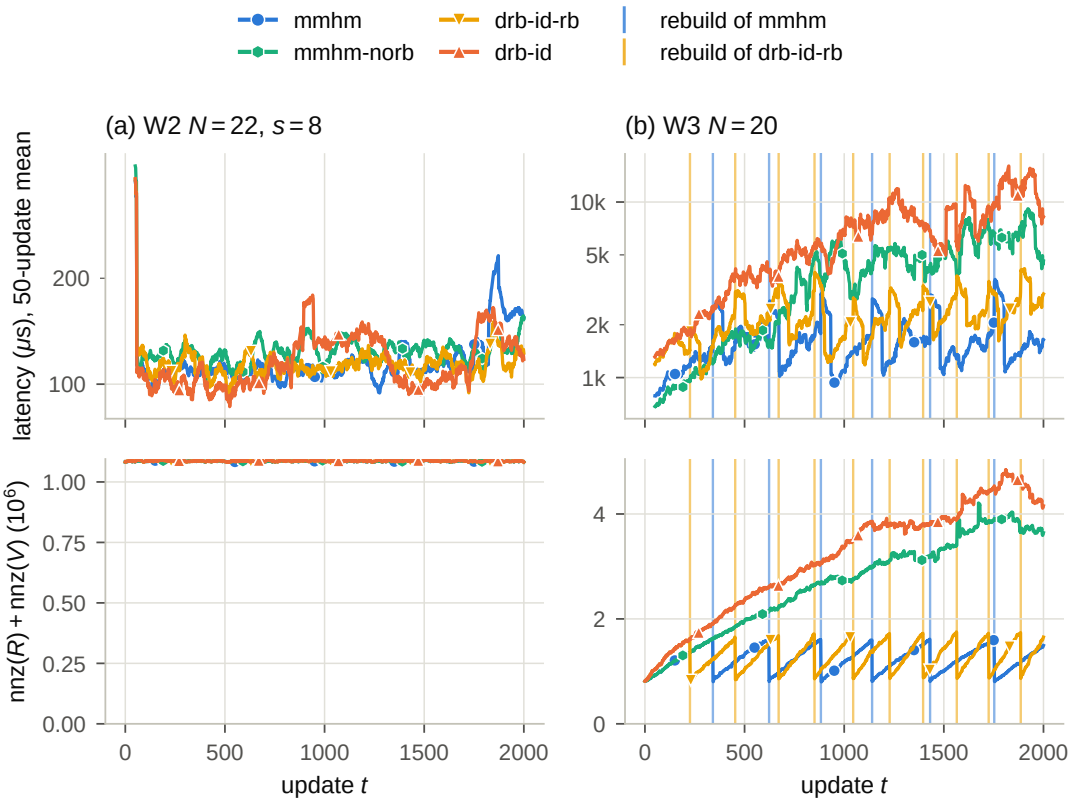


Figure 5: The four DRB variants over one run (seed 0). Top: per-update latency, mean over a sliding window of 50 updates. Bottom: $\mathrm{nnz}(R) + \mathrm{nnz}(V)$. Thin vertical lines through both panels: the rebuilds of mmhm (blue) and of drb-id-rb (yellow), each aligned with the drop of $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ it causes. (a) W2, $N = 22$, $s = 8$: the basis does not grow and no rebuild fires; (b) W3, $N = 20$: the basis grows until a rebuild resets it. The same plot for a W1 torus: Figure S3.

Table 5: The four DRB variants: amortised latency in ms (three runs pooled). Bold: fastest; °: within noise of the fastest. Rebuilds: mean number per run of drb-id-rb / mmhm under the default policy. The parameter sweep of the policy: Section S7.

instance	drb-id	mmhm-norb	drb-id-rb	mmhm	rebuilds
W2 $N = 33$, $s = 8$	0.165°	0.163°	**0.163**	0.165°	0 / 0
W3 $N = 20$	6.49	3.82	2.18	**1.57**	10 / 6.0
W3 $N = 27$	4.60	2.92	2.50	**1.81**	4.3 / 3.0
W4 2D $n = 16$k, $D = 2$	0.169	**0.110**	0.168	0.127	0 / 0
W1 sphere $L = 7$	37.1	**15.5**	39.2	16.6°	9.0 / 2.3
W1 torus 500×500	45.1	**33.4**	57.6	34.0°	1.7 / 1.0
W5a $L = 1{,}000{,}000$	14.2°	20.3	**14.0**	20.1	0 / 0
W5b level 8	**21.5**	29.4	21.7°	28.7	0 / 0

6.4 Ablations and algorithmic limits

6.4.1 When do the Morse order and rebuilding help?

Table 5 sets the four DRB variants side by side; together with the static reference of Table 4 it separates three effects.

The dynamic basis. Tracking the change of basis is what makes incremental maintenance possible, and on W2 and W4 it carries most of the gain: in insertion order and without rebuilds, drb-id is 88.7 times faster than the static reference on the largest W2 instance and 71.0 times on the largest W4 instance (where the best variant reaches 109).

Rebuilding. Rebuilding pays off where the basis grows. On W3 it is the largest single effect: at $N = 20$, switching the growth trigger on ($\alpha = 1$) takes the insertion-order DRB from 6,440 to 2,220 µs and the Morse-ordered one from 3,830 to 1,580 µs amortised. Figure 7 shows how this gain depends on how often the basis is rebuilt. The first few rebuilds per run remove most of the excess cost; beyond about ten per run the amortised latency of drb-id-rb changes

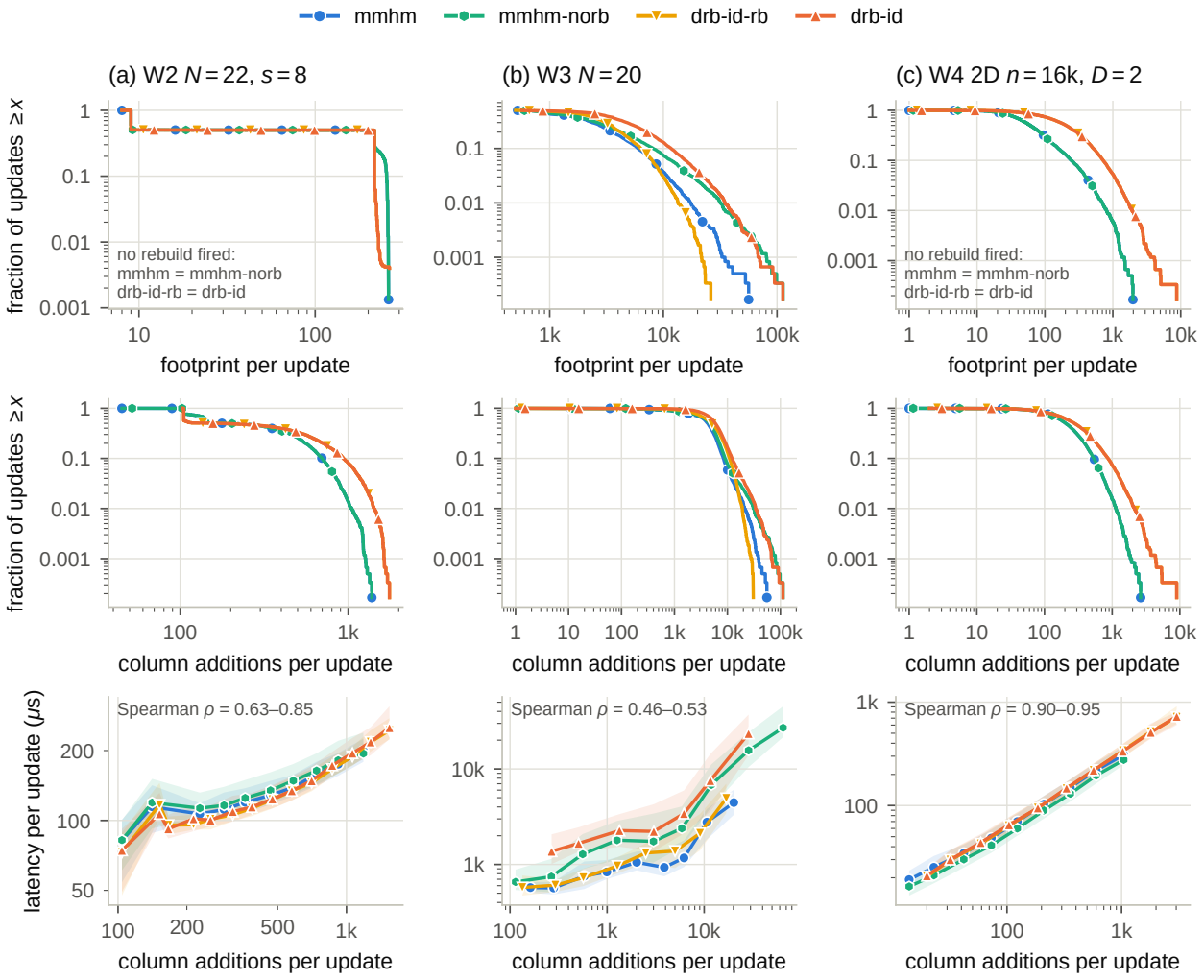


Figure 6: The cost proxies of Proposition 3 on one instance per family (updates without a rebuild, three runs pooled). Top: complementary cumulative distribution of the footprint of an update, the sum of $|\operatorname{occ}(\sigma)|$ over its deleted simplices; on W3 about half of the updates delete nothing. Middle: the same for the column additions of an update. Bottom: latency against column additions per update, median (line) and interquartile range (band) in logarithmic bins of at least 20 updates, with the range of Spearman's ρ over the four variants. The proxies count the terms of the sums in Proposition 3, not the sizes of the chains, and at a fixed count the latency still varies. (a) W2, $N = 22$, $s = 8$; (b) W3, $N = 20$; (c) W4 in the square, $n = 16000$, $D = 2$. On W2 and W4 no rebuild fired, so the proxies of mmhm equal those of mmhm-norb, and those of drb-id-rb those of drb-id. Quantiles: Table S11.

little, and that of mmhm rises again, from its value at the default to 1,960 µs with a rebuild every 64 updates. The tail is more sensitive. Once more than one update in a hundred carries a rebuild, the p99 is set by the rebuilds themselves: at $\alpha = 0.25$ the p99 of mmhm is 40,300 µs, against 6,710 µs at the default and 9,710 µs for recomputation (full-sorted). An aggressive policy is therefore counterproductive even where rebuilding helps. Where the basis does not grow (W2, W4) the trigger does not fire, and forced periodic rebuilds only add cost: every 64 updates, they raise the amortised latency of mmhm on W2 from 123 to 654 µs. On the surfaces of W1, rebuilds help on the smaller instances but not on the largest (Table S8).

The Morse order. Lemma 2 and its corollaries say where the work of a build is done; they do not make it smaller than in a good insertion order. Our generators write structured insertion orders, and at build time the Morse order saves almost nothing against them: on W2 it avoids the 192 additions of the insertion order (0 instead), on W1, W3 and W5 both orders need the same number of additions within one percent, and on W4 the Morse order needs 1.04 to 1.15 times as many (Table S6). What matters at build time is that the order is structured: in a random order (drb-rand) the basis has 1.84 to 2.29 times as many entries after the build, and updates on W1–W3 are 1.50 to 3.94 times slower than in insertion order (Table S14). Between rebuilds the Morse order helps where the basis grows or carries large cycles: mmhm-norb is faster than drb-id on W3 with $N \geq 20$, on W4 in the square with $n \geq 4000$ and on the large W1

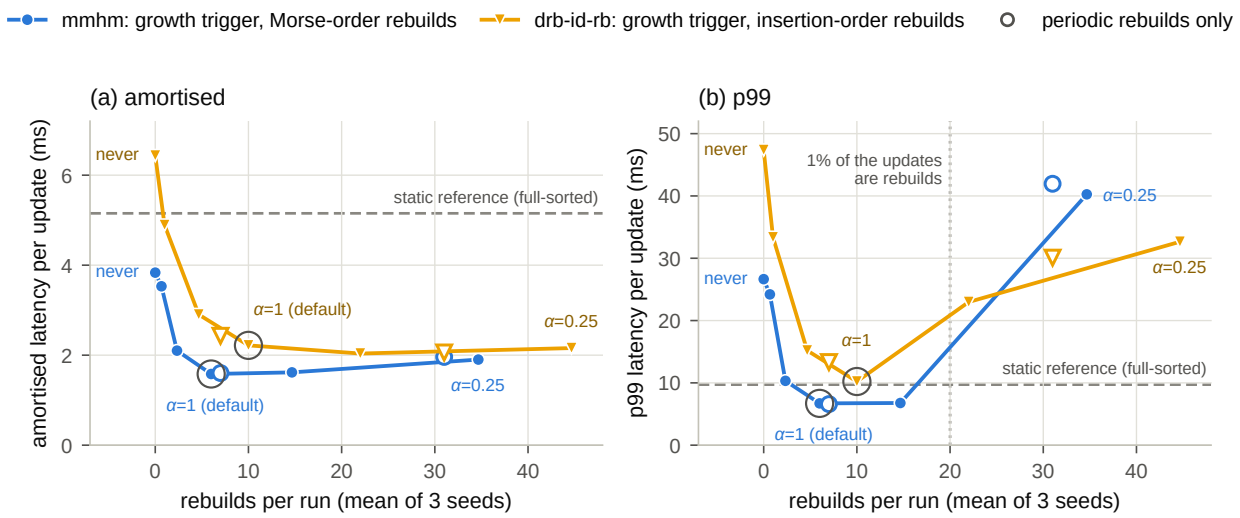


Figure 7: The rebuild policy on W3 with $N = 20$ (E7; three runs of $T = 2000$ updates, pooled): (a) amortised and (b) p99 latency per update, rebuilds included, against the number of rebuilds per run. Lines: the growth trigger alone, with $\alpha = \infty$ (never), 4, 2, 1, 0.5 and 0.25 from left to right; ring: the default $\alpha = 1$. Hollow markers: periodic rebuilds alone, every 256 and every 64 updates. Dashed: the static reference on the same instance (full-sorted, E3). Dotted, in (b): 20 rebuilds per run, beyond which more than 1 % of the updates carry a rebuild. The full sweep on W2, W3 and W4: Figure S4 and Table S5.

surfaces, within noise on W2, and slower on the W5 witnesses (Table 5). Without rebuilds it also shortens the tail of the column additions on W2, W3 and W4, and that of the footprints on W3 and W4 (Figure 6); after rebuilds on W3, the insertion-order rebuilds of drb-id-rb leave the lighter tails, although mmhm is faster (Section S8). At a rebuild it helps on W3, where mmhm is faster than drb-id-rb in 18 of the 18 settings of the policy sweep. The combination of growth-triggered rebuilds and the Morse order, mmhm, is the fastest variant on W3 at every size; on W4, where nothing triggers a rebuild, mmhm-norb is. There mmhm, which did not rebuild either, is slower than mmhm-norb beyond noise; we have not profiled this difference (Section S4.4).

6.4.2 Where does locality break down?

Non-locality witnesses. On W5 every toggle creates or destroys a cycle that spans the complex, and Proposition 4 forces every DRB state to do work linear in its size. The measurements follow it: the amortised latency per toggle grows with slope 1.06 to 1.10 on the cycle ($L = 10^3$ to 10^6) and 1.10 to 1.16 on the sphere (Figure 8), for every order and rebuild policy. Recomputation is linear as well, so the DRB remains faster on average (Table 4), but its p99 is higher on the cycle (at $L = 10^6$, 101,000 against 48,100 µs for mmhm) and, for the Morse-ordered variants, on the largest sphere (1.13 times that of full-id for mmhm). The bound concerns this representation, which stores cycle representatives explicitly; it is not a lower bound for dynamic Betti numbers. On a graph, for instance, β_0 and β_1 follow from dynamic connectivity in polylogarithmic amortised time [25].

Surfaces. W1 shows a related effect on a workload with local edits. Opening the first hole in a closed surface destroys a 2-cycle that spans the surface (Proposition 4), and the median latency of every DRB variant grows with the complex almost like that of recomputation (slopes 0.80 to 1.01 against 1.02 on the sphere); we did not log the chain sizes that would explain this. The amortised speedup over the static reference ranges from 0.592 to 1.98, and every DRB variant has a higher p99 than the static reference on every W1 instance, by factors 2.02 to 6.81. For connected surfaces with known connectivity, combinatorial counters give the Betti numbers in $O(1)$ per edit without any matrix (Remark 5.1). The full W1 results are in Section S5.

7 Discussion and limitations

Where the method is effective. The dynamic reduced basis replaces recomputation by work on the chains that an edit touches. When those chains stay small, as on refinement, carving and the flag complexes, the latency of an update depends little on the size of the complex and the gain over recomputation grows with it. When an edit creates or destroys a cycle that spans the complex, the work is linear in that cycle (Proposition 4), recomputation is linear too, and only a constant factor remains, with a heavier tail. Our five workload families mark these two regimes but do not settle which properties of a workload place it in one or the other.

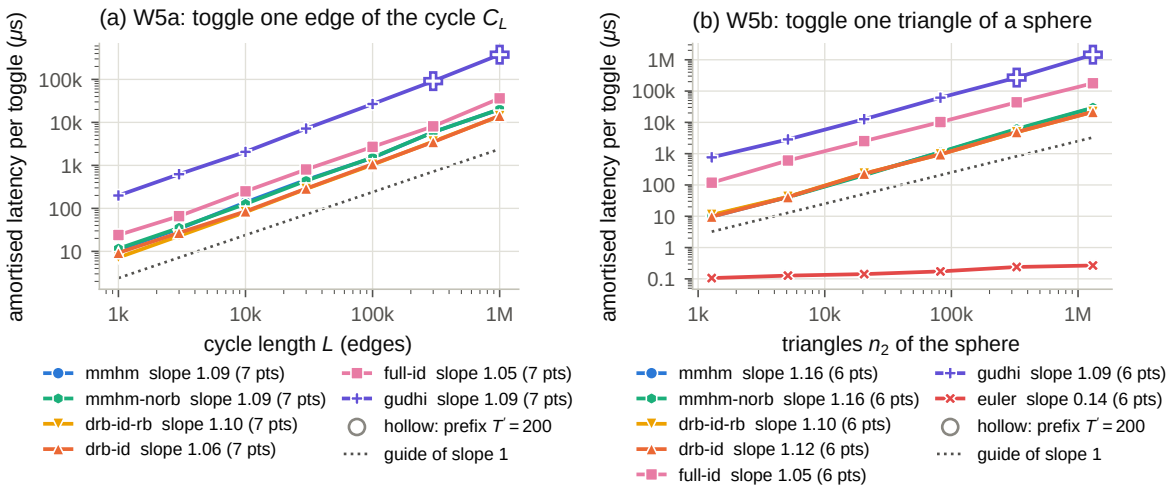


Figure 8: Amortised latency per toggle on the non-locality witnesses (log–log, three runs pooled; the legend gives each least-squares slope). (a) W5a against the cycle length L; (b) W5b against the number n_2 of triangles. Dotted: a guide of slope 1. Hollow markers: gudhi on the prefix T'. The linear growth of the DRB variants is what Proposition 4 forces for this representation, not a lower bound for the problem: the surface counters euler (Remark 5.1), which apply only to connected closed surfaces, stay below one microsecond in (b).

Basis sparsity and rebuilding. The cost of an update is governed by the size of the maintained basis, and on workloads whose edits change the topology the basis can fill in over time. Rebuilding resets it; a growth-triggered policy fired only where fill-in occurred and paid off there, whereas rebuilds on a basis that does not grow are pure overhead. Even where rebuilding pays off, rebuilding too often raises the tail latency above that of recomputation (Figure 7). The threshold and period are heuristics, and no setting was best on every workload.

The role of the Morse order. The order of the cells does not affect correctness (Theorem 1), and Section 4 shows what the Morse order guarantees at a build: every gradient pair is apparent and the work concentrates in the critical columns. Empirically, a random order made updates much slower on surfaces and refinement, more than the choice between two structured orders; on cavities and flag complexes the two effects were of similar size. The Morse order improved on insertion order between rebuilds and at rebuilds where the basis grows or carries large cycles, and not elsewhere. We make no sparsity claim for the state between rebuilds, and maintaining the matching under edits, which would extend the guarantees of Section 4 to all times, remains open.

Limitations. All results are over $\mathbb{Z}_2$: ranks over a field do not see torsion, and Betti numbers over $\mathbb{Z}_2$ can differ from those over other fields (the real projective plane has $(1, 1, 1)$ over $\mathbb{Z}_2$ and $(1, 0, 0)$ over the rationals [24]). The proofs hold in every dimension, but the engine and the experiments are restricted to dimension at most three. The cost of an update is bounded only in output-sensitive terms, and what we log are proxies for those terms. All timings come from one virtual machine, with single-threaded runs and one compiler setting, and memory was not measured. The baselines are static: we compared with recomputation and with GUDHI, but not with implementations of zigzag persistence or warm-started factorisations, which also support insertions and deletions and compute richer output [8, 28]. Our measurements therefore show an advantage over recomputation, not over these dynamic algorithms.

Alternatives. Specialised structures are preferable where their assumptions hold: fully dynamic connectivity for β_0, and for β_1 of graphs [25]; the incremental algorithm of Delfinado and Edelsbrunner for insertion-only complexes in the 3-sphere [11]; combinatorial counters for connected surfaces of known connectivity (Remark 5.1); and zigzag persistence when the barcode of the whole sequence is needed [7]. For small complexes, or when the tail latency matters on workloads with spanning cycles, static recomputation with clearing remains competitive. The dynamic reduced basis is suited to edit-local updates of large general complexes whose Betti numbers are needed online.

8 Related work

Static homology and persistence computation. Over a field, Betti numbers are ranks of boundary matrices, and the standard way to obtain them, together with the persistence barcode of a filtration, is column reduction in a fixed total order of the cells: each column receives additions from earlier columns until its lowest non-zero entry (its *low*) differs

from that of every other non-zero column [17, 18, 39]. The worst case is cubic in the number of cells [17], and practical performance comes from a small set of devices. *Clearing* (the *twist*) processes dimensions from the top down and sets to zero every column whose cell is already known to be the low of a reduced column one dimension up [9]; *clear and compress* pairs columns locally in chunks and compresses the remainder before a global reduction [3]. *Apparent pairs* are decided by a purely local condition on the boundary matrix, need no column additions, and are both persistence pairs and gradient pairs of a discrete Morse function; Ripser combines them with clearing, cohomology and an implicit coboundary [1]. PHAT [4] and GUDHI [30] package these variants, and Otter et al. [36] benchmark the available software. All of this is static: an edited complex is a new input. Our rebuild step is exactly such a static reduction with clearing, modified to retain the change of basis V; our full-* baselines rerun it, without V, after every update.

Discrete Morse theory and coreductions. Forman's discrete Morse theory [20, 21] replaces a complex by a chain-homotopy-equivalent Morse complex on the critical cells of an acyclic matching. Algebraic Morse theory [26, 38] extends the construction to chain complexes of free modules and obtains the Morse complex by eliminating the matched pairs, whose incidences are invertible. Algorithmically, matchings are built by collapses or coreductions and used to shrink the input before any linear algebra: coreduction homology [33], extended to regular CW complexes [16], to maps and persistence [23, 34], and filtered Morse complexes that preserve persistent homology [32]. On 2-manifolds, matchings with the minimum number of critical cells can be constructed in linear time [27]; in general, Benedetti and Lutz [5] collapse a free face chosen at random and declare a cell critical only when no free face is left, and use the resulting distribution of critical-cell counts to measure how complicated a triangulation is. Our matching is a deterministic variant of their collapse strategy. We use it differently from the works above: we never form the Morse complex, whose boundary entries are sums over gradient paths, so that an edit to a *matched* cell can change the boundaries of critical cells far from the edit. Instead we use the matching only to choose the total order of the *unreduced* boundary matrix, so that every gradient pair is an apparent pair. This step is elementary and close to known results: the collapse removes each paired coface immediately before its face, so the two are adjacent in the reverse removal order, and a facet and a coface that are adjacent in a filtration order always form an apparent pair. In the other direction, the apparent pairs of any simplexwise filtration form a discrete gradient [1], and Bauer and Roll [2] relate the gradient pairs of a (generalised) discrete Morse function to the zero-persistence apparent pairs of a lexicographically refined filtration order. What we add is the consequence for the static reduction with clearing: all column additions happen in the columns of critical cells, and on critical rows and columns the reduction performs exactly the reduction of the Morse complex (Corollaries 2.1 and 2.2).

Updating persistence. Vineyards [10] maintain the decomposition $R = DV$ of a filtered boundary matrix, with V stored, under transpositions of adjacent cells at linear cost per transposition. Our dynamic reduced basis (DRB) is the same kind of object, a reduced matrix $R = BV$ with V invertible and tracked, maintained under a different set of moves. Zigzag persistence [7] handles both insertions and deletions. Carlsson, de Silva and Morozov [8] gave an incremental algorithm; Milosavljević, Morozov and Škraba [31] bounded the cost of n additions and deletions by matrix-multiplication time; Maria and Schreiber [29] use discrete Morse reductions as preprocessing, with cost depending mostly on the number of critical cells; and FastZigzag [13] converts a simplex-wise zigzag filtration into an ordinary filtration of a Δ-complex of the same length, so that standard persistence software applies. On graphs, zigzag persistence takes near-linear time using dynamic connectivity structures [12], and vineyard updates extend to zigzag filtrations, including expansions and contractions [14]. Dey, Fan and Wang [15] maintain $\mathbb{Z}_2$ simplex annotations under elementary inclusions and vertex collapses to compute persistence along simplicial maps. Closest to our deletion step are methods that warm-start from an existing $R = DV$ factorisation when a filtration changes. Luo and Nelson [28] extend the vineyard updates to cell insertions and deletions processed in a batch, with cost scaling with the number of elementary changes. Giunti and Lazovskis [22] update R, V and the representative cycles when a simplex and its cofaces are removed, compatibly with the twist (clearing) optimisation.

Incremental and dynamic Betti numbers. Delfinado and Edelsbrunner [11] compute the Betti numbers of a complex in the 3-sphere built by adding one simplex at a time, in $O(n\,\alpha(n))$ time. Each new simplex either creates a cycle or destroys one (the positive/negative dichotomy that is also our insertion step); in S^3 the test reduces to union–find on the complex and on its complement, with β_1 following from the Euler characteristic. The algorithm is insertion-only and depends on the embedding. Fully dynamic β_0 under edge insertions and deletions is dynamic graph connectivity, solved in polylogarithmic amortised time per update [25]; for graphs embedded on a surface, Eppstein [19] maintains generators of the surface's fundamental group under edge insertions, deletions and their duals. These bounds rely on special structure (a graph, an ambient S^3, an embedding surface) and none of them covers β_1, β_2 and β_3 of an arbitrary complex of dimension at most three under insertions and deletions, which is our setting.

Positioning. Our dynamic reduced basis maintains $R_k = B_k V_k$ with V_k tracked under column insertion and column deletion, which suffices because rows appear and disappear only while they are zero. The deletion procedure is not new

in substance: it is the rank-only specialisation of the deletion updates used for zigzag persistence and for warm-started $R = DV$ factorisations [8, 22, 28]; we give a self-contained proof. The difference lies in the output and in the freedom it buys. Zigzag persistence yields the barcode of the entire sequence of complexes, a strictly richer output, and an incremental zigzag algorithm [8] already answers our query. In zigzag, vineyard and warm-start methods, however, the cell order is the filtration and is fixed by the input. We report only the current Betti numbers, so the order is a computational device that we may choose (the Morse order) and replace at a rebuild; its measured effect depends on the workload (Section 6.4.1). For β_0 alone, and for insertion-only complexes in S^3, the specialised structures above have better worst-case bounds.

9 Conclusion

Tracking the change of basis of a reduced boundary matrix makes it possible to maintain the $\mathbb{Z}_2$ Betti numbers of a simplicial complex exactly under insertions and deletions of simplices, for every order of the cells. Insertion is the standard reduction step, and deletion is the rank-only form of known zigzag and warm-start updates. A collapse order makes every gradient pair an apparent pair, so that a build performs its column additions only in critical columns, where they are exactly the reduction of the Morse complex. The cost of an update is output-sensitive, and on explicit families it is linear in the size of the complex for edits of size one.

Experimentally, the dynamic basis is exact on every state we checked and, where edits create and destroy only small cycles, much cheaper than recomputation: on our largest refinement instance every variant is 88.6–89.9 times faster than the better static baseline. This gain comes from the basis itself. Rebuilding controls the growth of the basis where edits change the topology, and the Morse order helps on some of these workloads; their combination is the fastest variant on cavities. Where edits create cycles that span the complex, the work is linear, as the theory predicts, and the method keeps at most a constant factor over recomputation, with a heavier tail.

Open questions are maintaining the matching between rebuilds, rebuild policies with guarantees, bounds on footprints and reduction chains for structured families of complexes and edits, other coefficients, and comparisons with dynamic algorithms such as zigzag persistence and warm-started factorisations.

Code and data. Code, workloads, raw results and the scripts that regenerate every number and figure are available from the author. The file `experiments/README.md` describes how to regenerate them, and `experiments/results/WORKLOADS_MANIFEST.csv` records the generator command and the SHA-256 of every workload instance.

References

[1] Ulrich Bauer. Ripser: Efficient computation of Vietoris–Rips persistence barcodes. *Journal of Applied and Computational Topology*, 5(3):391–423, 2021. doi:10.1007/s41468-021-00071-5.

[2] Ulrich Bauer and Fabian Roll. Wrapping cycles in Delaunay complexes: Bridging persistent homology and discrete Morse theory. In Wolfgang Mulzer and Jeff M. Phillips, editors, *40th International Symposium on Computational Geometry (SoCG 2024)*, volume 293 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 15:1–15:16, 2024. doi:10.4230/LIPIcs.SoCG.2024.15.

[3] Ulrich Bauer, Michael Kerber, and Jan Reininghaus. Clear and compress: Computing persistent homology in chunks. In *Topological Methods in Data Analysis and Visualization III*, pages 103–117. Springer, 2014. doi:10.1007/978-3-319-04099-8_7.

[4] Ulrich Bauer, Michael Kerber, Jan Reininghaus, and Hubert Wagner. PHAT – persistent homology algorithms toolbox. *Journal of Symbolic Computation*, 78:76–90, 2017. doi:10.1016/j.jsc.2016.03.008.

[5] Bruno Benedetti and Frank H. Lutz. Random discrete Morse theory and a new library of triangulations. *Experimental Mathematics*, 23(1):66–94, 2014.

[6] Gunnar Carlsson. Topology and data. *Bulletin of the American Mathematical Society*, 46:255–308, 2009. doi:10.1090/S0273-0979-09-01249-X.

[7] Gunnar Carlsson and Vin de Silva. Zigzag persistence. *Foundations of Computational Mathematics*, 10:367–405, 2010. doi:10.1007/s10208-010-9066-0.

[8] Gunnar Carlsson, Vin de Silva, and Dmitriy Morozov. Zigzag persistent homology and real-valued functions. In *Proceedings of the Symposium on Computational Geometry (SoCG 2009)*, pages 247–256, 2009. doi:10.1145/1542362.1542408.

[9] Chao Chen and Michael Kerber. Persistent homology computation with a twist. In *Proceedings of the 27th European Workshop on Computational Geometry (EuroCG 2011)*, pages 197–200, Morschach, Switzerland, 2011.

[10] David Cohen-Steiner, Herbert Edelsbrunner, and Dmitriy Morozov. Vines and vineyards by updating persistence in linear time. In *Proceedings of the Symposium on Computational Geometry (SoCG 2006)*, pages 119–126, 2006. doi:10.1145/1137856.1137877.

[11] Cecil Jose A. Delfinado and Herbert Edelsbrunner. An incremental algorithm for Betti numbers of simplicial complexes on the 3-sphere. *Computer Aided Geometric Design*, 12(7):771–784, 1995. doi:10.1016/0167-8396(95)00016-Y.

[12] Tamal K. Dey and Tao Hou. Computing zigzag persistence on graphs in near-linear time. In *37th International Symposium on Computational Geometry (SoCG 2021)*, volume 189 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 30:1–30:15, 2021. doi:10.4230/LIPIcs.SoCG.2021.30.

[13] Tamal K. Dey and Tao Hou. Fast computation of zigzag persistence. In *European Symposium on Algorithms (ESA 2022)*, volume 244 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 43:1–43:15, 2022. doi:10.4230/LIPIcs.ESA.2022.43.

[14] Tamal K. Dey and Tao Hou. Computing zigzag vineyard efficiently including expansions and contractions. In *40th International Symposium on Computational Geometry (SoCG 2024)*, volume 293 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 49:1–49:15, 2024. doi:10.4230/LIPIcs.SoCG.2024.49.

[15] Tamal K. Dey, Fengtao Fan, and Yusu Wang. Computing topological persistence for simplicial maps. In *Proceedings of the 30th Annual Symposium on Computational Geometry (SoCG 2014)*, 2014.

[16] Paweł Dłotko, Tomasz Kaczynski, Marian Mrozek, and Thomas Wanner. Coreduction homology algorithm for regular CW-complexes. *Discrete & Computational Geometry*, 46(2):361–388, 2011. doi:10.1007/s00454-010-9303-y.

[17] Herbert Edelsbrunner and John L. Harer. *Computational Topology: An Introduction*. American Mathematical Society, Providence, RI, 2010. ISBN 978-0-8218-4925-5. doi:10.1090/mbk/069.

[18] Herbert Edelsbrunner, David Letscher, and Afra Zomorodian. Topological persistence and simplification. *Discrete & Computational Geometry*, 28(4):511–533, 2002. doi:10.1007/s00454-002-2885-2.

[19] David Eppstein. Dynamic generators of topologically embedded graphs. In *Proceedings of the 14th ACM-SIAM Symposium on Discrete Algorithms (SODA 2003)*, pages 599–608, 2003.

[20] Robin Forman. Morse theory for cell complexes. *Advances in Mathematics*, 134:90–145, 1998.

[21] Robin Forman. A user's guide to discrete Morse theory. *Séminaire Lotharingien de Combinatoire*, 48:Art. B48c, 2002.

[22] Barbara Giunti and Jānis Lazovskis. Pruning vineyards: Updating barcodes and representative cycles by removing simplices. *Foundations of Data Science*, 2026. doi:10.3934/fods.2026007. Early access; preprint arXiv:2312.03925.

[23] Shaun Harker, Konstantin Mischaikow, Marian Mrozek, and Vidit Nanda. Discrete Morse theoretic algorithms for computing homology of complexes and maps. *Foundations of Computational Mathematics*, 14:151–184, 2014. doi:10.1007/s10208-013-9145-0.

[24] Allen Hatcher. *Algebraic Topology*. Cambridge University Press, Cambridge, 2002.

[25] Jacob Holm, Kristian de Lichtenberg, and Mikkel Thorup. Poly-logarithmic deterministic fully-dynamic algorithms for connectivity, minimum spanning tree, 2-edge, and biconnectivity. *Journal of the ACM*, 48(4):723–760, 2001. doi:10.1145/502090.502095.

[26] Michael Jöllenbeck and Volkmar Welker. Minimal resolutions via algebraic discrete Morse theory. *Memoirs of the American Mathematical Society*, 197(923), 2009.

[27] Thomas Lewiner, Hélio Lopes, and Geovan Tavares. Optimal discrete Morse functions for 2-manifolds. *Computational Geometry*, 26(3):221–233, 2003.

[28] Yuan Luo and Bradley J. Nelson. Accelerating iterated persistent homology computations with warm starts. *Computational Geometry*, 2024. doi:10.1016/j.comgeo.2024.102089. Preprint arXiv:2108.05022, 2021.

[29] Clément Maria and Hannah Schreiber. Discrete Morse theory for computing zigzag persistence. *Discrete & Computational Geometry*, 71(2):708–737, 2024. Conference version in WADS 2019.

[30] Clément Maria, Jean-Daniel Boissonnat, Marc Glisse, and Mariette Yvinec. The Gudhi library: Simplicial complexes and persistent homology. In *Mathematical Software – ICMS 2014*, volume 8592 of *Lecture Notes in Computer Science*, pages 167–174. Springer, 2014.

[31] Nikola Milosavljević, Dmitriy Morozov, and Primož Škraba. Zigzag persistent homology in matrix multiplication time. In *Proceedings of the 27th Annual Symposium on Computational Geometry (SoCG 2011)*, pages 216–225, 2011. doi:10.1145/1998196.1998229.

[32] Konstantin Mischaikow and Vidit Nanda. Morse theory for filtrations and efficient computation of persistent homology. *Discrete & Computational Geometry*, 50(2):330–353, 2013. doi:10.1007/s00454-013-9529-6.

[33] Marian Mrozek and Bogdan Batko. Coreduction homology algorithm. *Discrete & Computational Geometry*, 41: 96–118, 2009.

[34] Marian Mrozek and Thomas Wanner. Coreduction homology algorithm for inclusions and persistent homology. *Computers & Mathematics with Applications*, 60(10):2812–2833, 2010.

[35] James R. Munkres. *Elements of Algebraic Topology*. Addison-Wesley, Menlo Park, CA, 1984.

[36] Nina Otter, Mason A. Porter, Ulrike Tillmann, Peter Grindrod, and Heather A. Harrington. A roadmap for the computation of persistent homology. *EPJ Data Science*, 6:17, 2017. doi:10.1140/epjds/s13688-017-0109-5.

[37] Udo Pachner. P.L. homeomorphic manifolds are equivalent by elementary shellings. *European Journal of Combinatorics*, 12(2):129–145, 1991. doi:10.1016/S0195-6698(13)80080-7.

[38] Emil Sköldberg. Morse theory from an algebraic viewpoint. *Transactions of the American Mathematical Society*, 358(1):115–129, 2006.

[39] Afra Zomorodian and Gunnar Carlsson. Computing persistent homology. *Discrete & Computational Geometry*, 33(2):249–274, 2005. doi:10.1007/s00454-004-1146-y.